Mahidol University
Wisdom of the Land

ITCS215
Introduction to Interactive Multimedia Technology

Class: 1-2026
Module III: Introduction to Computer Graphics, Generative AI, and Audio
Lecture: TUE 13:00-16:00

Unit 4: Music, Acoustic Analysis Tools, and Emotion Recognition

Instructor: Lecturer Dr. Kanoksak Wattanachote, PhD (金功勳博士)
Contact: kanoksak.wat@mahidol.ac.th

Grading

ITCS215 Introduction to Interactive Multimedia Technology

- Overall Score Parts
  - Class activities + Assignment 30%
  - Quizzes 30%
  - Final Exam 40%
- Module 1 (5 weeks): Introduction to Animation, Interactive Application, Text, and Font.
- Module 2 (5 weeks)**:** Introduction to Image Analysis and Machine Learning.
- **Module 3** (5 weeks): **Introduction to Computer Graphics, Generative AI, Sound-Visualization/Analytics**.
  - Class activities + Assignment 10%
  - Quizzes 10%
  - Final Exam 40/3 = 13.33%

Unit 4: Content

Unit 4: MediaPipe, Video, and Sound Technologies

- Music
- Vocal Extraction
- Sound Analysis Tools
- File Format
- Music Information
- Sound Emotion Recognition
- Feature Extraction

Acoustic

Displaying using librosa

Python in Jupyter Notebook

- Connect to Google Drive

```
from google.colab import drive
ROOT = '/content/gdrive'
drive.mount(ROOT, force_remount=True)
# do only once
!ln -s /content/gdrive/My\ Drive/ /mydrive
!ls /mydrive/Sounds-of-Music/Songs
```

```
Mounted at /content/gdrive
02Bitch.mp3  Outputs  pop.00021.wav  สรุปว่าบ้า.mp3
```

Python Library for Audio Files

- The character 'sr' in this code stands for "sampling rate," which is the number of audio samples per second.
- When we load an audio file with librosa.load(), it returns two things:
  - The audio time series data (y) and
  - The sampling rate (sr).

```
import librosa
from librosa import display
import numpy as np
import IPython.display as ipd
import matplotlib as plt

y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')
print(y)
print(sr)
ipd.Audio(data=y[90*sr:110*sr], rate=sr)
```



Python Library for Audio Files

- 90*sr and 110*sr define the start and end points in the sampling unit for a specific section of the audio:
- 90*sr: Specifies the sample index number at which the section of the audio should begin, which is the 90-second position. Multiplying by the sampling rate means converting 90 seconds to an equivalent sample index number.

```
import librosa
from librosa import display
import numpy as np
import IPython.display as ipd
import matplotlib as plt

y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')
ipd.Audio(data=y[90*sr:110*sr], rate=sr)
```

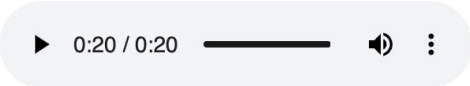


Python Library for Audio Files

- 110*sr: Similarly, this command specifies the sample index position where the audio segment should end at the 110-second mark.

```
import librosa
from librosa import display
import numpy as np
import IPython.display as ipd
import matplotlib as plt

y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')
ipd.Audio(data=y[90*sr:110*sr], rate=sr)
```



Python Library for Audio Files

- Therefore, y[90*sr:110*sr] extracts a portion of the audio corresponding to a time interval between 90 seconds and 110 seconds.
- This audio portion is then replayed using ipd.Audio with the playback rate set to the original sampling rate (sr).

```
import librosa
from librosa import display
import numpy as np
import IPython.display as ipd
import matplotlib as plt

y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')
ipd.Audio(data=y[90*sr:110*sr], rate=sr)
```



Spectrogram Visualization

- Mounting Drive

```
from google.colab import drive
ROOT = '/content/gdrive'
drive.mount(ROOT, force_remount=True)
# do only once
!ln -s /content/gdrive/My\ Drive/ /mydrive
!ls /mydrive/Sounds-of-Music/Songs
```

Spectrogram Visualization

- Display Spectrogram

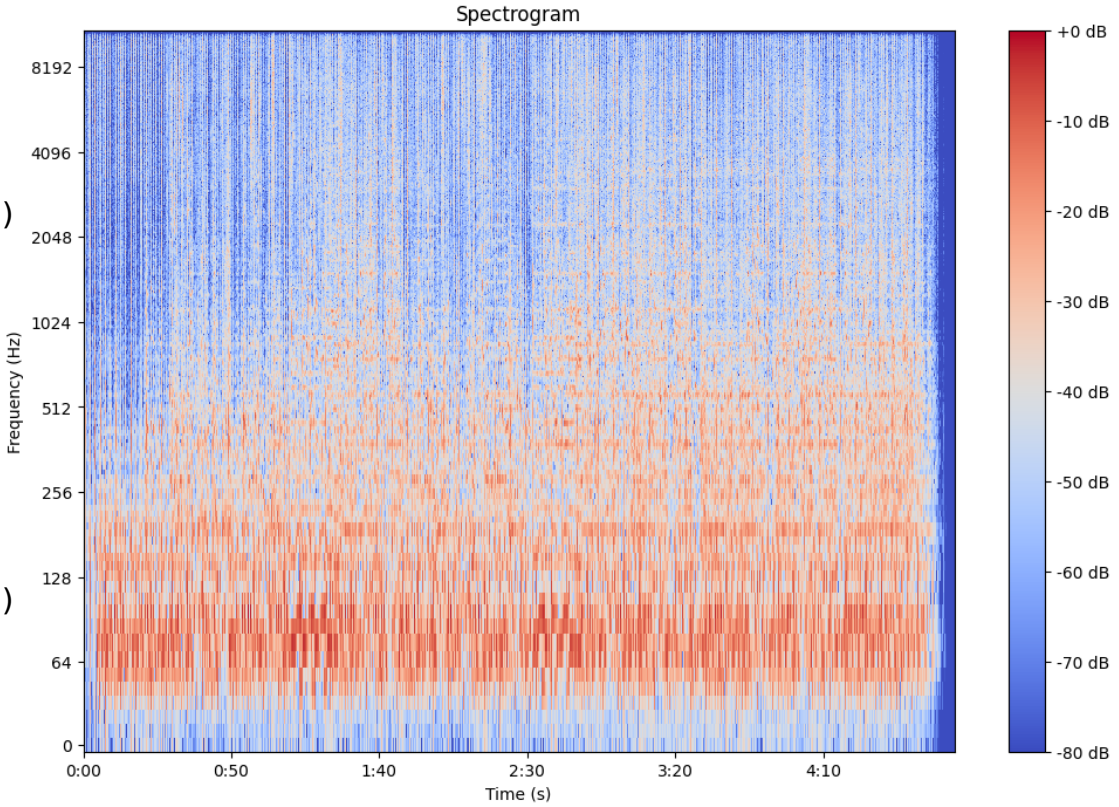
```
import matplotlib.pyplot as plt
import librosa.display
import numpy as np # Added for np.max

# Load the audio file
y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')

# Compute the STFT to get the spectrogram
D = np.abs(librosa.stft(y))

# Convert the amplitude spectrogram to a dB scale
D_dB = librosa.amplitude_to_db(D, ref=np.max)

# Plot the spectrogram
plt.figure(figsize=(12, 8))
librosa.display.specshow(D_dB, sr=sr, x_axis='time',
                         y_axis='log', cmap='coolwarm')
plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.show()
```



Waveform Visualization

- Display Waveform

```
import matplotlib.pyplot as plt
import librosa.display
import numpy as np

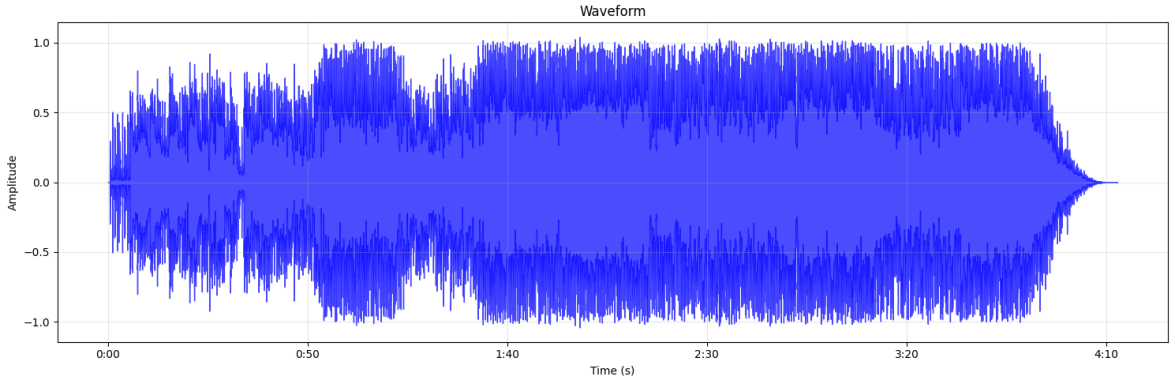
# Load the audio file
y, sr = librosa.load('/mydrive/Sounds-of-Music/Songs/02Bitch.mp3')

# Plot the waveform
plt.figure(figsize=(15, 5))

# ใช้ waveshow แสดง waveform
librosa.display.waveshow(y, sr=sr, alpha=0.7, color='blue')

plt.title('Waveform')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# แสดงข้อมูลเพิ่มเติมเกี่ยวกับไฟล์เสียง
print(f"ความยาวเสียง: {len(y)/sr:.2f} วินาที")
print(f"Sample rate: {sr} Hz")
print(f"จำนวน samples: {len(y)}")
print(f"แอมพลิจูดสูงสุด: {np.max(np.abs(y)):.3f}")
print(f"แอมพลิจูดเฉลี่ย: {np.mean(np.abs(y)):.3f}")
```




```
ความยาวเสียง: 252.84 วินาที
Sample rate: 22050 Hz
จำนวน samples: 5575104
แอมพลิจูดสูงสุด: 1.040
แอมพลิจูดเฉลี่ย: 0.169
```

Introduction
to

Music



Music

Module III: Music

- Music is the arrangement of sound to create some combination of form,
  - harmony,
  - melody,
  - rhythm, or
  - otherwise expressive content.
- Music is generally agreed to be a cultural universal that is present in all human societies.
- Definitions of music vary widely in substance and approach.

Harmony

Module III: Harmony

- In music, harmony is the concept of combining different sounds to create new, distinct musical ideas.
- Theories of harmony seek to describe or explain the effects created by distinct pitches or tones coinciding with one another.

Pitch

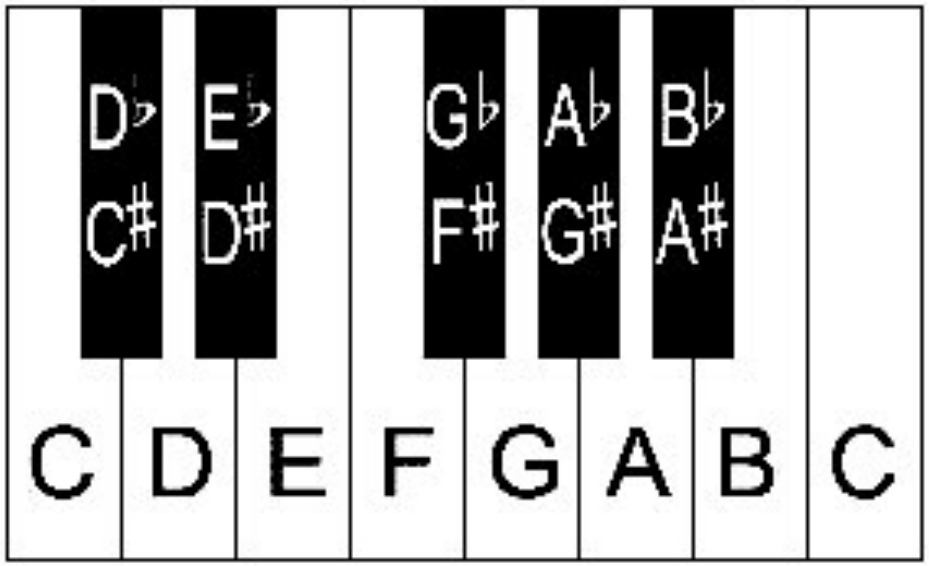
Module III: Pitch

- Pitch just means how high or low a sound is.
- Think about animal sounds:
  - A little mouse's squeak is high-pitched.
  - A big lion's roar is low-pitched.

Pitch

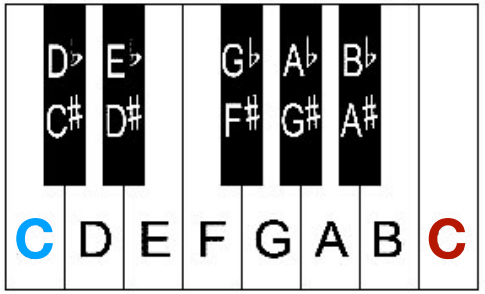
Module III: Pitch

- On a piano, the keys on the right side make high-pitched sounds (like the mouse).
- The keys on the left side make low-pitched sounds (like the lion).



Pitch

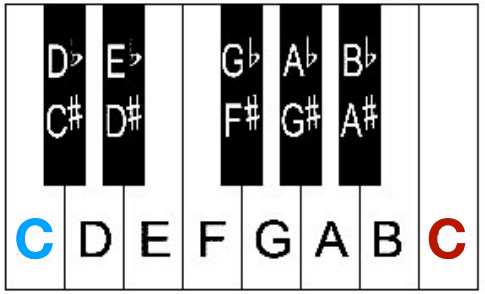
Module III: Pitch



- The musical alphabet is a short one — it only uses a handful of letters out of our English alphabet.
- It's the first seven letters, A, B, C, D, E, F, and G.
  - They go in the same order that we say them in...A...B...C and so on.
  - The tricky part is what to do when you get to G? — No, we don't just stop...we start over!
  - The notes would go something like this: A, B, C, D, E, F, G...A, B, C, and so on until you run out of piano keys.
- The 8th note C is referred to as one Octave above the first note C.
  - It also represents the first note of the next set of 7 notes, or octave.
- The reason the same note is reused is that the number one notes all sound the same, except that they are different pitches.

Pitch

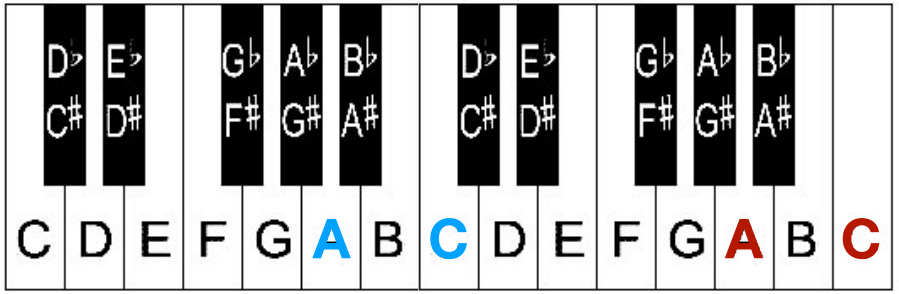
Module III: Pitch



- The white keys represent the ‘normal’ notes on a piano, like I was just describing above: A, B, C, D, E, F, and G are considered to be the normal, unaltered notes.
- The black keys represent the sharps and flats, or more commonly called accidentals, and are altered versions of the seven main notes.
- They are written to represent this fact, such as
  - D# for D-sharp, or
  - Ab to represent A-flat.

Pitch

Module III: Pitch

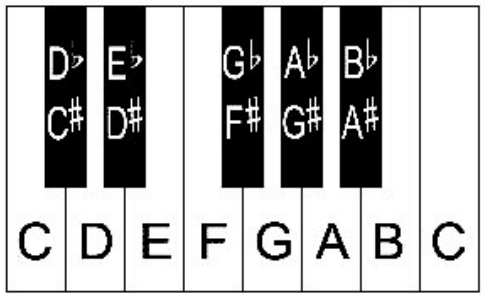


- Since pitch is the perceived highness or lowness of a sound, it is the raw material of music.
- Physically, pitch is determined by the frequency of a sound wave, measured in **Hertz** (Hz).
- A higher frequency, e.g., 440 Hz for the **A** above middle **C**, is perceived as a high pitch.
- A lower frequency, e.g., 110 Hz for the **A** two octaves lower, is perceived as a low pitch.

Pitch

Module III: Pitch

- A standard piano has 88 keys, covering a little over 7 octaves.
- This means there are multiple A notes, each an octave apart.
- Here are the main ones, from lowest to highest:

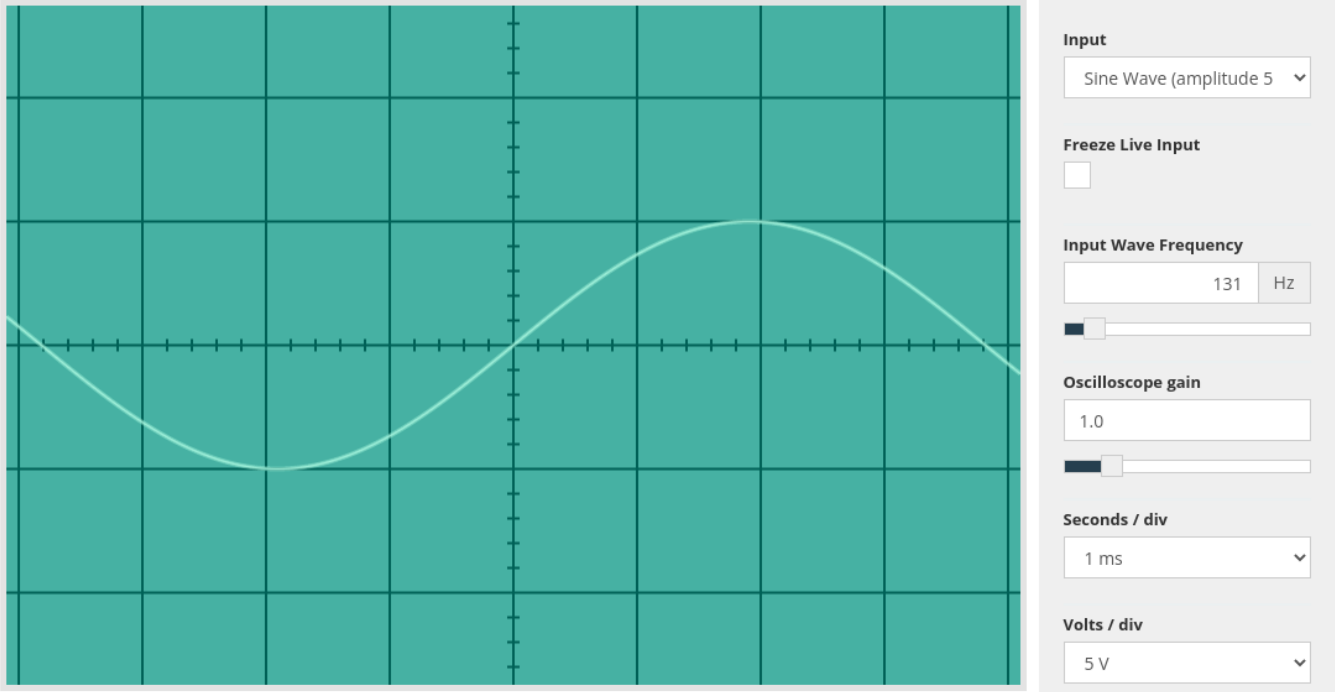


Note Name	Frequency	Description
A0	**27.5 Hz**	The lowest A on a standard 88-key piano. It sounds very deep and rumbling.
A1	**55 Hz**	
A2	**110 Hz**	The open A string on a guitar (which is tuned to A2, 110 Hz).
A3	**220 Hz**	The A below Middle C.
A4	**440 Hz**	The A above Middle C. This is our target note in the lecture and theory.
A5	**880 Hz**	One octave above A4.
A6	**1760 Hz**	
A7	**3520 Hz**	The highest A on a standard 88-key piano. It sounds very bright and piercing.

Pitch

Module III: Pitch

- Lower pitches have a lower frequency— the frequency is half that of middle **C** (**C4**) (131 Hz). (Scale: 1 square is equal to 1 millisecond)

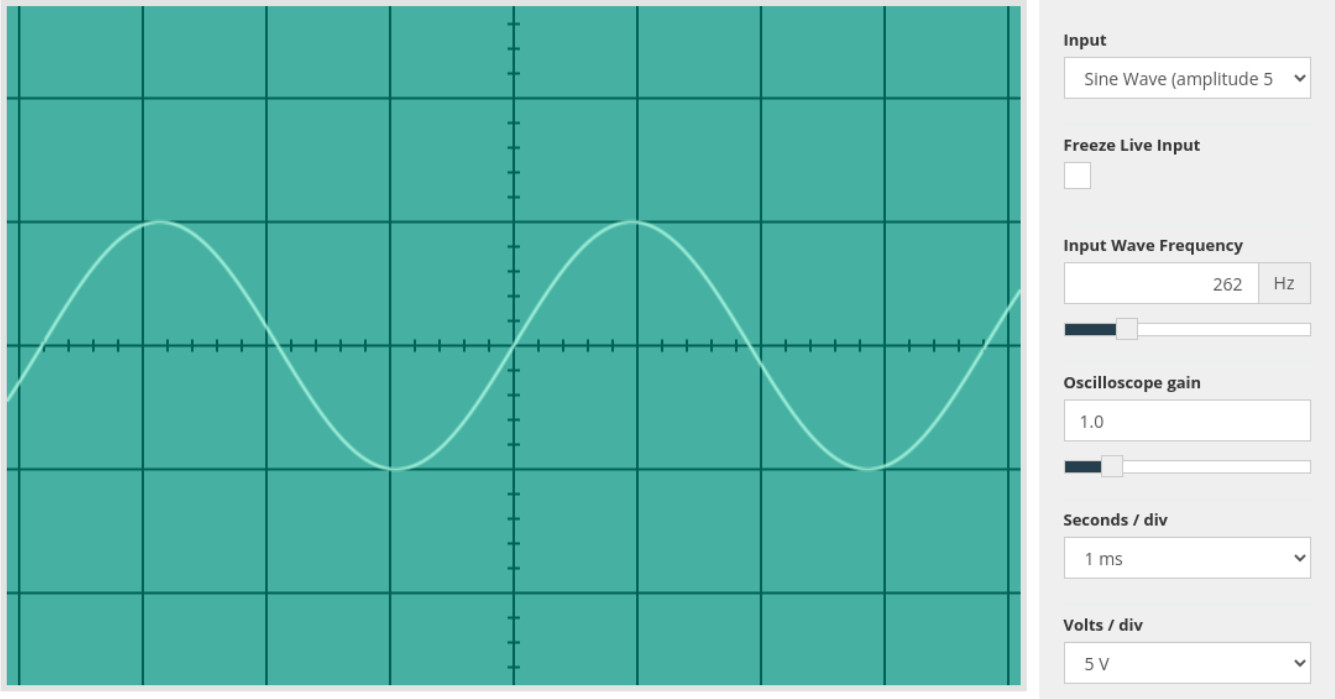


source: https://en.wikipedia.org/wiki/Pitch_(music)

Pitch

Module III: Pitch

- Oscillogram of middle **C** (**C4**) (approximately 261.63 Hz) (pure tone) — on the piano, if you find Middle C (C4).

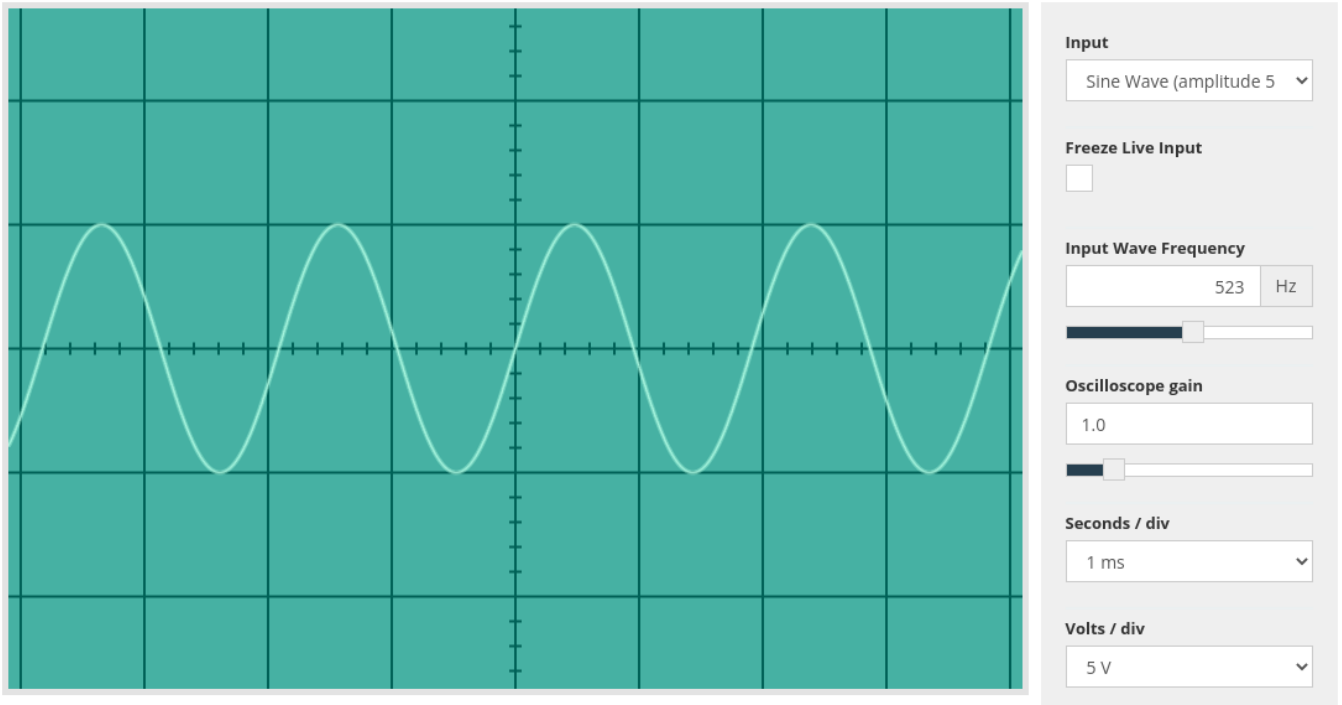


source: https://en.wikipedia.org/wiki/Pitch_(music)

Pitch

Module III: Pitch

- Higher pitches have higher frequency — oscillogram of **C5**, an octave above middle **C** — the frequency is twice that of middle C (523 Hz).

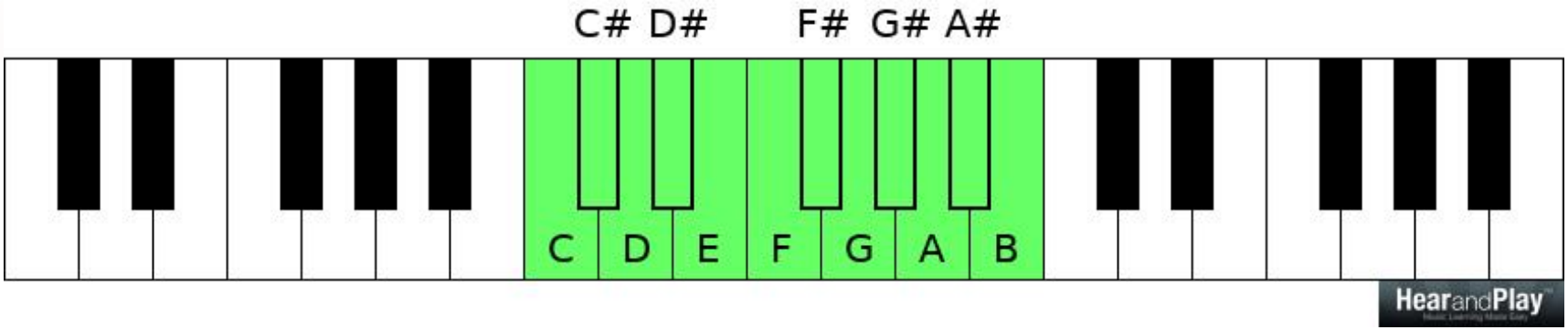


source: https://en.wikipedia.org/wiki/Pitch_(music)

Note

Module III: Note

- A **note** can be defined as any sound that belongs to any of the twelve pitch classes on the piano.
- It is a musical sound with a precise pitch.
- Here are the twelve of them:



Melody

Module III: Melody

- When a sequence of notes is connected in such a way that they are played or heard separately, the outcome is known as a **melody**.
- Even though **melody** is a sequence of notes, subjecting it to a detailed examination (aka "melodic analysis") will show you that it is deliberate, it has a sense of direction, and most importantly, a meaning.
    - Any idea that is an outcome of the relationship between notes that are heard at the same time is said to be **harmonic**.
    - Harmony is the outcome of the relationship.

Tone

Module III: Tone

- Tone is the “Color” or “Personality” of a Sound.
- Imagine two people saying the same word, like “hello.”
  - A deep, booming voice says “hello.”
  - A soft, whispery voice says “hello.”
- The word is the same (that’s like the pitch or note), but the sound of the voice is completely different.
- That difference in the sound’s quality is what musicians call **Tone** (or the fancy word: **Timbre**).

Tone

Module III: Tone

- Think of a single musical note as a scoop of vanilla ice cream.
  - It's pure and simple.
- But most sounds are more like a fancy sundae.
- The vanilla ice cream is the main note (the fundamental pitch), but it's covered in:
  - Chocolate syrup
  - Whipped cream
  - A cherry
  - Sprinkles
  - Nuts
- These extra toppings are like **overtones**.
- They are higher, quieter sounds that naturally happen on top of the main note.

Tone

Module III: Tone

- Every instrument has its own unique recipe of these "**overtones**."
  - A flute is like a scoop of plain vanilla ice cream.
    - It has very few overtones, so it sounds pure and smooth.
  - A guitar is like a sundae with chocolate syrup and nuts.
    - It has a richer, more complex tone.
  - A trumpet is like a sundae with everything on it.
    - It has lots of bright, brassy overtones.

- The unique combination of the main note + its **overtones** is what gives each instrument its special sound.

Meaning
of

Sound Waveform



Waveform

Module III: Sound Waveform

- `librosa.display.waveshow(y, sr=sr)` was used to display the waveform.
- The waveform's Y-axis is the amplitude of the digital audio samples, usually normalized between -1.0 and 1.0.
  - Values close to 0 → silence or very soft sound.
  - Values closer to ±1 → higher energy in the signal (louder parts).

```
# Pick one file from the folder
file_list = [f for f in os.listdir(folder) if f.endswith('.mp3')]
print("Files found:", file_list)

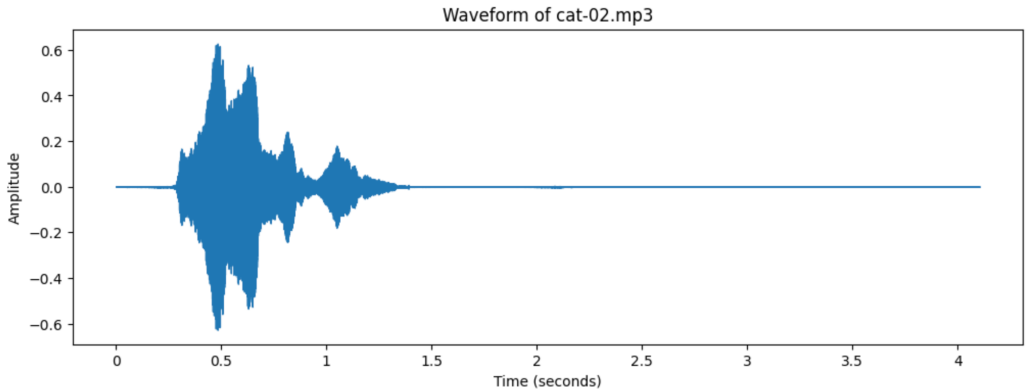
for fname in file_list:
    path = os.path.join(folder, fname)
    y, sr = librosa.load(path, sr=None)

    print(f"\nNow playing: {fname}")
    print(f"Duration: {len(y)/sr:.2f} seconds")

    # Plot
    plt.figure(figsize=(12, 4))
    librosa.display.waveshow(y, sr=sr)
    plt.title(f'Waveform of {fname}')
    plt.xlabel('Time (seconds)')
    plt.ylabel('Amplitude')
    plt.show()

    # Play sound
    display(ipd.Audio(path))
```

```
Now playing: cat-02.mp3
Duration: 4.10 seconds
```



0:00 / 0:04

Waveform

Module III: Sound Waveform

- Interpret the Waveform
  - Quiet Sections: Flat, near-zero amplitude lines → silence or very soft sound.
  - Loud Sections: Big peaks and troughs → louder parts of the audio.
  - Patterns: Repeating oscillations can indicate tones, speech, or musical notes.

```
# Pick one file from the folder
file_list = [f for f in os.listdir(folder) if f.endswith('.mp3')]
print("Files found:", file_list)

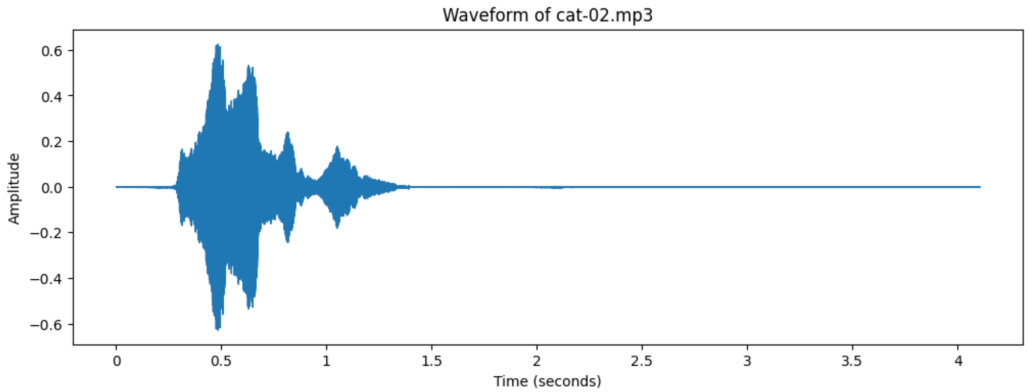
for fname in file_list:
    path = os.path.join(folder, fname)
    y, sr = librosa.load(path, sr=None)

    print(f"\nNow playing: {fname}")
    print(f"Duration: {len(y)/sr:.2f} seconds")

    # Plot
    plt.figure(figsize=(12, 4))
    librosa.display.waveshow(y, sr=sr)
    plt.title(f'Waveform of {fname}')
    plt.xlabel('Time (seconds)')
    plt.ylabel('Amplitude')
    plt.show()

    # Play sound
    display(ipd.Audio(path))
```

```
Now playing: cat-02.mp3
Duration: 4.10 seconds
```



0:00 / 0:04

Music

Vocal Extraction

Vocal Extraction: Demucs

- Installation
  - demucs

```
!pip install demucs -qq
```

Vocal Extraction: Demucs

- Preview
  - Separate vocals and instruments.

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

import os

# Define paths
input_audio = '/content/drive/MyDrive/Sounds-of-Music/Songs/02Bitch.mp3'
output_directory = '/content/drive/MyDrive/Sounds-of-Music/Outputs/'

# Create output directory
os.makedirs(output_directory, exist_ok=True)

# Use demucs via command line (most reliable method)
!demucs --two-stems=vocals "{input_audio}" -o "{output_directory}"

print("Separation complete!")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Important: the default model was recently changed to `htdemucs` the latest Hybrid Transformer Demucs model. In some cases, this model ca
Selected model is a bag of 1 models. You will see that many progress bars per track.
Separated tracks will be stored in /content/drive/MyDrive/Sounds-of-Music/Outputs/htdemucs
Separating track /content/drive/MyDrive/Sounds-of-Music/Songs/02Bitch.mp3
 23%|                                                                    | 58.5/257.4 [02:30<09:02,  2.73s/seconds]
```

Vocal Extraction: Demucs

- Preview
  - Separate vocals and instruments.

```
import os
from IPython.display import Audio, display

input_audio = '/content/drive/MyDrive/Sounds-of-Music/Songs/02Bitch.mp3'
output_dir = '/content/drive/MyDrive/Sounds-of-Music/Outputs/htdemucs/'
base = os.path.splitext(os.path.basename(input_audio))[0]

# หาโฟลเดอร์ผลลัพธ์
sep_folder = os.path.join(output_dir, base)
if not os.path.exists(sep_folder):
    sep_folder = os.path.join(output_dir, 'hta', base)

if os.path.exists(sep_folder):
    print(f"ไฟล์ต้นฉบับ:")
    display(Audio(input_audio))

    for file in ['vocals.wav', 'no_vocals.wav', 'drums.wav', 'bass.wav', 'other.wav']:
        path = os.path.join(sep_folder, file)
        if os.path.exists(path):
            print(f"\n {file}:")
            display(Audio(path))
else:
    print("ไม่พบโฟลเดอร์ผลลัพธ์")
```

```
... ไฟล์ต้นฉบับ:
▶ 0:00 / 4:12
vocals.wav:
▶ 0:00 / 4:12
no_vocals.wav:
▶ 0:00 / 4:12
```

Vocal Extraction: Demucs

- Functions
  - Types of separation

```
!demucs                          # demucs (w/o arguments) for 4 tracks
                                 # 4 tracks: contain vocals, drums, bass, and other
!demucs --two-stems=vocals       # 2 tracks: vocals and no-vocals
!demucs -n htdemucs_6s           # 6 tracks: vocals, drums, bass, piano, guitar, other
```

Vocal Extraction: Demucs

- Pretrained models
  - The characteristics of the types

```
# Hybrid Transformer Demucs (default/present)
!demucs -n htdemucs "file.mp3"

# MDX Extra (old)
!demucs -n mdx_extra_q "file.mp3"

# MDX (another)
!demucs -n mdx_q "file.mp3"

# Demucs V3 (old)
!demucs -n demucs "file.mp3"

# Demucs V2 (older)
!demucs -n demucs_v2 "file.mp3"

# Lightweight version
!demucs -n lightweight “file.mp3"
```

Model	Speed	Quality	Model Size
htdemucs (default)	Medium	Best	~1.2 GB
mdx_extra_q	Slow	Very Good	~1.5 GB
mdx_q	Slow	Good	~1.3 GB
demucs	Fast	Medium	~500 MB
demucs_v2	Fast	Medium	~500 MB
lightweight	Very Fast	Fair / Acceptable	~200 MB

Practices

- How to select a portion of an audio file for analysis purposes?
- How to separate the vocal ranges of a song?
- Then compare and describe the two vocal ranges with explanations.
- Display the results using a Spectrogram.
- Display the results using a Waveform.

Practices

- Separate speech/vocal sounds from sound effects/music.
- Then compare and explain the two sounds: human voice/vocal and sound effects/music, providing an explanation.
- Display the results using a spectrogram.
- Also display the results using a waveform.

- Select data from the Kaggle link in the slide, then choose two sentences (each containing at least six emotions). Observe the spectrogram and waveform data to determine if it's possible to use this information as a tool to differentiate emotions from words containing these six emotions.

Introduction
to

Sound Analysis Tools



MFCC

Module III: Sound Analysis Tool

- MFCC stands for Mel-Frequency Cepstral Coefficients, a widely used feature extraction technique in audio and speech processing.
- It transforms a raw audio signal into a set of numerical values (features) that capture the important characteristics of the sound, particularly those relevant to human hearing.

	mfcc_1_mean	mfcc_2_mean	mfcc_3_mean	mfcc_4_mean	mfcc_5_mean	mfcc_6_mean	mfcc_7_mean	mfcc_8_mean	mfcc_9_mean	mfcc_10_mean	...	mfcc_20_mean	spectral_centroid_mean	label
0	-22.110100	-3.660113	-0.027843	0.835313	0.008838	-0.449172	-0.240558	-0.395879	-0.135695	-0.418983	...	-0.293790	5501.114233	cat
1	-21.948895	-3.562472	0.132024	0.332999	-0.116666	-0.330796	0.362622	0.107612	-0.393424	-0.437474	...	-0.109152	5505.678360	cat
2	-22.346428	-4.525930	-0.280700	0.348921	0.923535	-0.033493	0.023169	-0.166009	-0.223189	0.234156	...	0.098386	5527.618027	cat
3	-21.417542	-3.266622	0.097575	0.520383	0.503650	0.099866	-0.418804	-0.472339	0.077510	-0.039472	...	0.008917	5501.347195	dog
4	-22.607563	-4.391189	-0.851798	-0.040459	0.207845	-0.323180	0.246994	0.171743	0.250802	-0.265807	...	0.066552	5518.976644	dog
5	-21.524992	-3.342219	0.623189	0.509321	-0.084219	-1.258322	-0.583415	0.639359	-0.073305	-0.406962	...	0.291729	5513.187772	dog

6 rows × 22 columns

MFCC

Module III: Sound Analysis Tool

- The human ear does not perceive all frequencies equally—it is more sensitive to certain ranges (like speech frequencies between ~300 Hz and ~3400 Hz).
- MFCC simulates this perception using the Mel scale, making the extracted features more relevant for tasks such as:
    - Speech recognition
    - Speaker identification
    - Music genre classification
    - Environmental sound classification

MFCC

Module III: Sound Analysis Tool

- Steps in MFCC Extraction
  1. Pre-emphasis
     - Boosts high-frequency components to balance the spectrum.
  2. Framing
     - Splits the signal into small overlapping frames, e.g., 20–40 ms, to analyze short-term features.
  3. Windowing
     - Applies a window function, like Hamming, to reduce edge discontinuities.

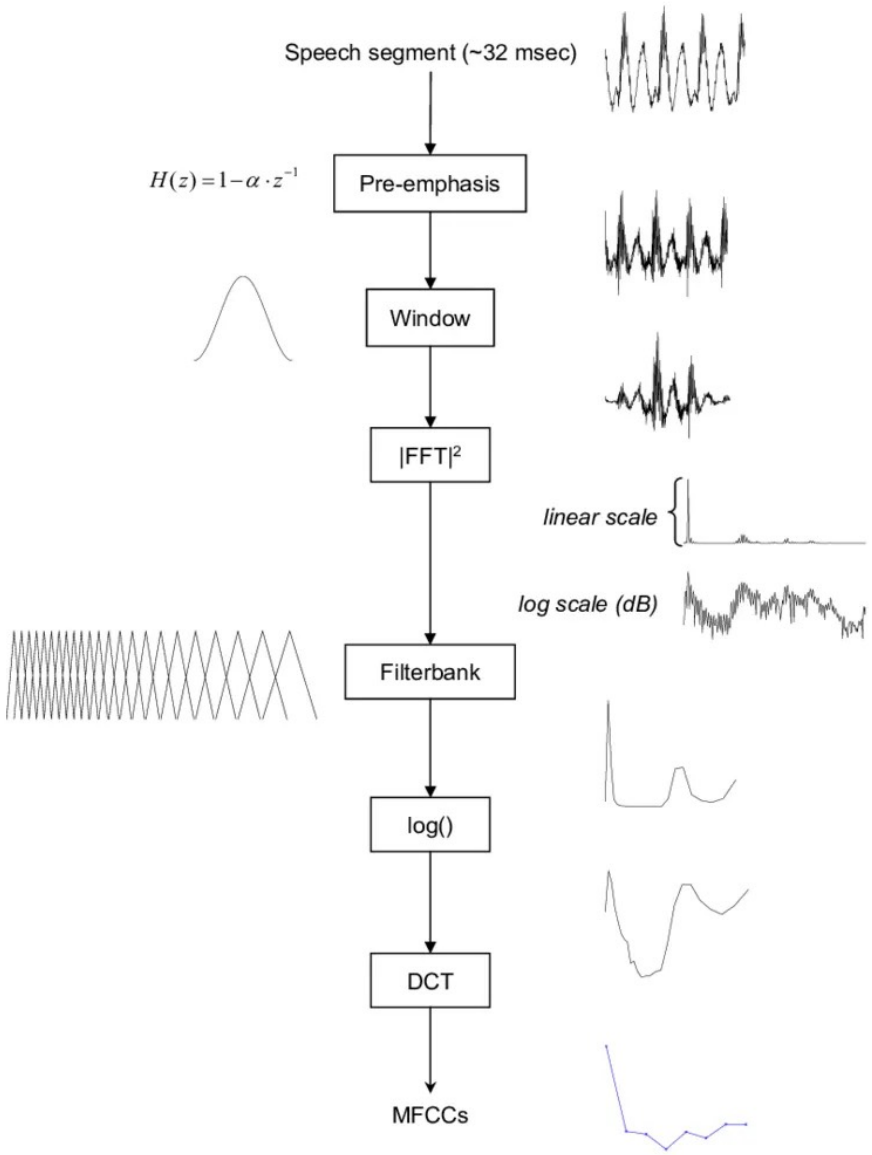


Photo Credit: https://dr-pathasu-doung.medium.com/mel-frequency-cepstrum-coefficient-mfcc-b735a349c312

MFCC

Module III: Sound Analysis Tool

- Steps in MFCC Extraction
  4. Fast Fourier Transform (FFT)
     - Converts each frame from the time domain to the frequency domain.
  5. Mel Filter Bank
     - Passes the spectrum through triangular filters spaced according to the Mel scale to emphasize perceptually important frequencies.
  6. Logarithm
     - Converts amplitude to a logarithmic scale to match human loudness perception.

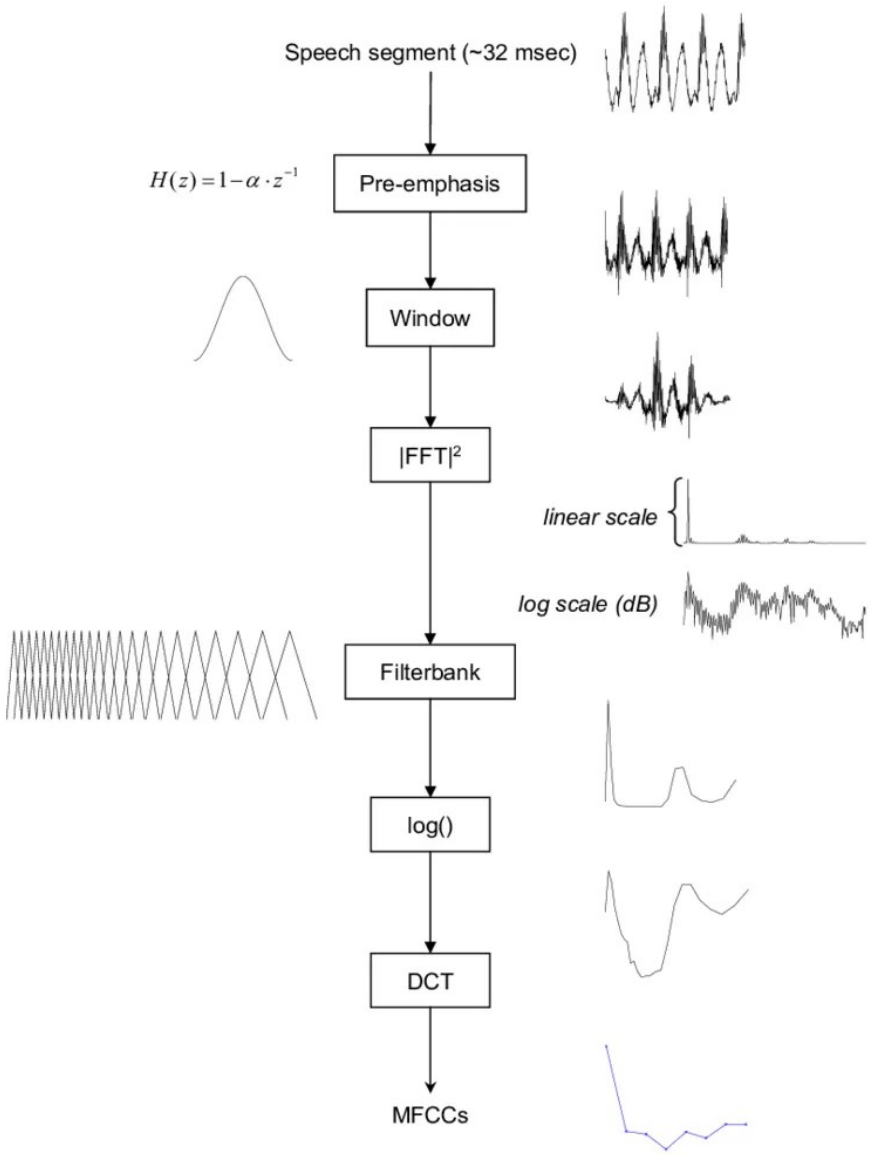


Photo Credit: https://dr-pathasu-doung.medium.com/mel-frequency-cepstrum-coefficient-mfcc-b735a349c312

MFCC

Module III: Sound Analysis Tool

- Steps in MFCC Extraction
  7. Discrete Cosine Transform (DCT)
     - Produces a small set of coefficients, e.g., first 12–13 values, that represent the overall spectral shape of the sound.

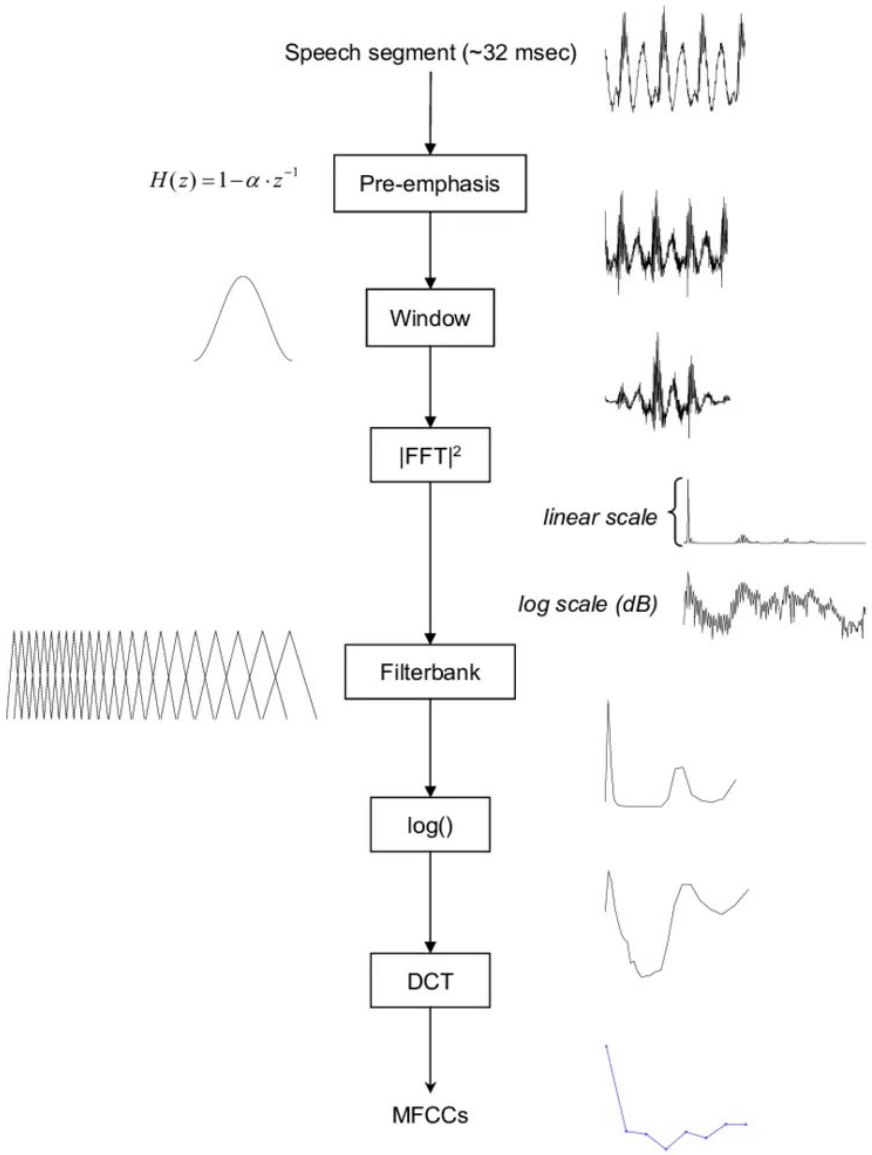


Photo Credit: https://dr-pathasu-doung.medium.com/mel-frequency-cepstrum-coefficient-mfcc-b735a349c312

MFCC

Module III: Sound Analysis Tool

- Typical Structure

Column	Meaning
`mfcc1_mean` → `mfcc20_mean`	Average values of the 20 MFCC coefficients over the audio duration
`spectral_centroid_mean`	Brightness of the sound
`spectral_bandwidth_mean`, `rms_mean`	Additional audio descriptors
`label`	The class or tag for that audio clip (e.g., `speech`, `music`, `noise`)

- 13 coefficients: Standard for speech recognition (captures the vocal tract features well).
- 20 coefficients: Captures more detail in the spectral envelope, which can be helpful in:
  - Music analysis
  - Environmental sound classification
  - Emotion recognition in speech

Audio

File Format



Lossy Formats

Module III: Sound and Digital Audio

- Lossy audio formats lose data in the transmission — do not decompress back to their original file size, so they end up smaller, and some sound waves are lost.
  - **MP3** (MPEG-1 Audio Layer III) is the most popular of the lossy formats — the files can be as small as one-tenth the size of lossless files.
  - **AAC**, Advanced Audio Coding (known as MPEG-4 AAC), takes up very little space and is good for streaming, over mobile devices — requiring less than 1 MB per minute of music, sounding better than MP3 at the same bitrate, used by iTunes/ Apple Music, YouTube, and Android.

Lossless Formats

Module III: Sound and Digital Audio

- These files decompress back to their original size, keeping sound quality intact — audio professionals want all of the original sound waves, so they prefer lossless.
  - **FLAC**, a Free Lossless Audio Codec, offers lossless compression and it's free and open-source.
  - **ALAC**, Apple's Lossless Audio Codec, allows for lossless compression, but it works only on Apple devices.

Uncompressed Formats

Module III: Sound and Digital Audio

- These files remain the same size from origin to destination.
  - **WAV**, Waveform Audio File, retains all the original data, which makes it the ideal format for sound engineers — it's the highest quality.
  - **AIFF**, Audio Interchange File Format, created by Apple, files are like WAV files in that they retain all of the original sound and take up more space than MP3s.
    - It can play on Macs and PCs.
    - They don't hold time codes, so they're not as useful for editing and mixing.

Digital Audio Practice

Music Information



Working with Shared Resource

Module III: Sound and Digital Audio

- Select some music files for a project by adding some files into the subfolder “Songs” in “My Drive”.

```
from google.colab import drive
ROOT = '/content/gdrive'
drive.mount(ROOT, force_remount=True)
# do only once
!ln -s /content/gdrive/My\ Drive/ /mydrive
!ls /mydrive
!ls /mydrive/Songs
```

Working with Shared Resource

Module III: Sound and Digital Audio

- Librosa is a popular Python library used primarily for audio and music analysis.
- It provides tools that make it easy to work with audio data:
  - **Loading Audio Files**: The “librosa” can load various audio file formats, e.g., WAV and MP3, and convert them into NumPy arrays for easy processing in Python.

```
!pip install librosa matplotlib IPython
```

  - **Audio Manipulation**: The “librosa” can be used to resample, trim, and slice audio, making it easy to work with specific parts of an audio file.
  - **Feature Extraction**: The “librosa” can extract features from audio, e.g., Spectograms, Mel-frequency cepstral coefficients (MFCCs), Chroma features, and tempo and beat tracking, for transcription, classification, and more.

Working with Shared Resource

Module III: Sound and Digital Audio

- Librosa is a popular Python library used primarily for audio and music analysis.
- It provides tools that make it easy to work with audio data:
  - **Pitch and Tempo Analysis**: It provides functions for analyzing the pitch, tempo, and rhythm of audio, which is particularly useful for music and speech processing tasks.

```
!pip install librosa matplotlib IPython
```

  - **Visualization**: The “librosa” can be combined with Matplotlib to visualize audio data, such as waveforms, spectrograms, and other features, making it easier to analyze and understand the structure of the audio.

Working with Shared Resource

Module III: Sound and Digital Audio

- The “librosa” is widely used in various fields involving audio data, such as:
  - Music Information Retrieval (MIR)
  - Audio Analysis
  - Machine Learning Application.

```
!pip install librosa matplotlib IPython
```

Python Library for Audio Files

Module III: Sound and Digital Audio

- The ‘sr’ in this code stands for “sampling rate”, which is the number of samples of audio per second.
- When we load an audio file with librosa.load(), it returns two things:
  - the audio time series data (y) and
  - the sampling rate (sr).

```
import librosa
from librosa import display
import numpy as np
import IPython.display as ipd
import matplotlib as plt

y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')
ipd.Audio(data=y[90*sr:110*sr], rate=sr)
```



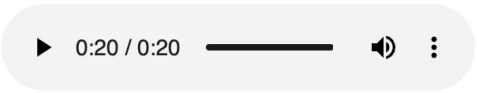
Python Library for Audio Files

Module III: Sound and Digital Audio

- The 90*sr and 110*sr are defining the start and end points in samples for a specific segment of the audio:
  - 90*sr: This specifies the sample index where the audio segment should start, which is the 90-second mark — multiplying by the sampling rate means converting 90 seconds into the equivalent sample index.

```
import librosa
from librosa import display
import numpy as np
import IPython.display as ipd
import matplotlib as plt

y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')
ipd.Audio(data=y[90*sr:110*sr], rate=sr)
```



Python Library for Audio Files

Module III: Sound and Digital Audio

- The 90*sr and 110*sr are defining the start and end points in samples for a specific segment of the audio:
  - 110*sr: Similarly, this specifies the sample index where the audio segment should end, at the 110-second mark.

```
import librosa
from librosa import display
import numpy as np
import IPython.display as ipd
import matplotlib as plt

y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')
ipd.Audio(data=y[90*sr:110*sr], rate=sr)
```



Python Library for Audio Files

Module III: Sound and Digital Audio

- Therefore, y[90*sr:110*sr] extracts a slice of the audio corresponding to the time interval from 90 seconds to 110 seconds.
- This segment of the audio is then played back using ipd.Audio, with the playback rate set to the original sampling rate (sr).

```
import librosa
from librosa import display
import numpy as np
import IPython.display as ipd
import matplotlib as plt

y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')
ipd.Audio(data=y[90*sr:110*sr], rate=sr)
```



Spectrogram Visualization

Module III: Sound and Digital Audio

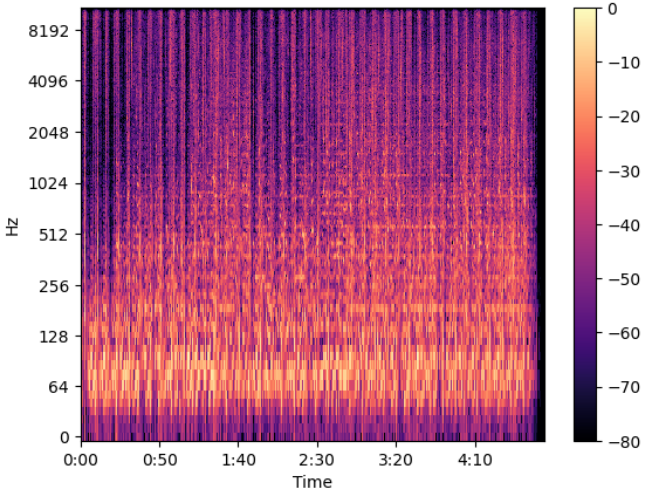
- Challenge:
  - How to select a part of the audio file for some analysis purposes.
  - How to separate the vocal (for transcribing and text mining analysis) and accompaniment (for song genre analysis and instrument components analysis) of music.

Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- This code is an example of how to create and display a spectrogram using Librosa and Matplotlib.
- It uses magphase to separate the magnitude and phase information from the Short-Time Fourier Transform (STFT) of an audio signal.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                        ref=np.max), y_axis='log', x_axis='time',
                        sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

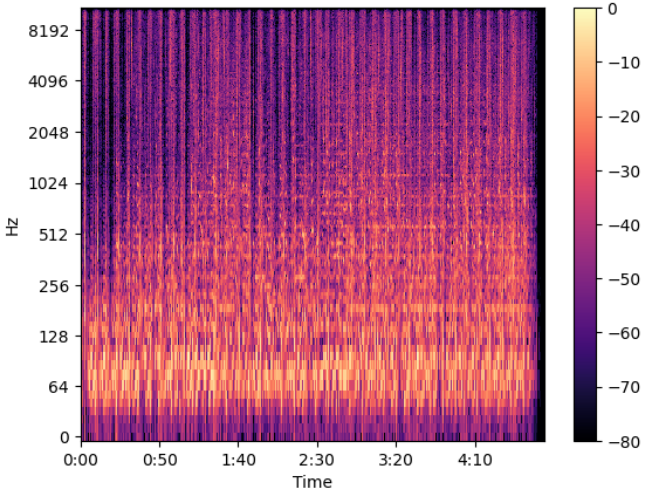


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- S_full: represents the magnitude component of the STFT of the audio signal.
  - librosa.stft(y) computes the STFT, producing a complex matrix where each element corresponds to a frequency bin and time window.
  - librosa.magphase() separates this complex matrix into magnitude and phase components, since S_full contains the magnitudes of the frequency bins, which is a key part of the spectrogram visualization.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

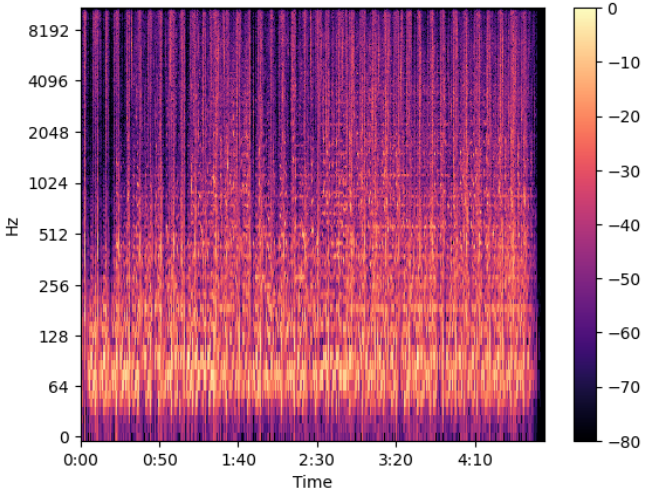


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- phase: holds the phase information of the STFT output.
  - It is not used directly in the spectrogram plot but can be useful for reconstructing the original signal if needed.
  - librosa.magphase() returns the magnitude and phase of the STFT, where phase contains only the phase angles of each frequency bin.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

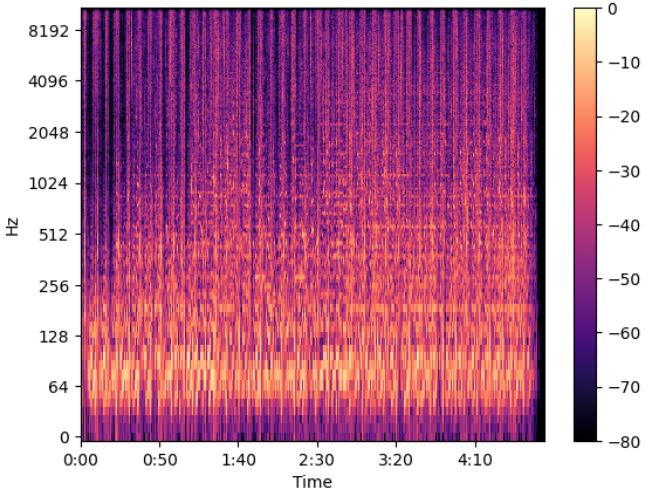


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- idx: defines the range of time frames to display in the spectrogram.
  - librosa.time_to_frames([90, 110], sr=sr) converts the time range [90, 110] seconds into frame indices based on the sample rate, sr.
  - slice(*librosa.time_to_frames([90, 110], sr=sr)) generates a slice object, which selects the time frames from 90 to 110 seconds in the spectrogram.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

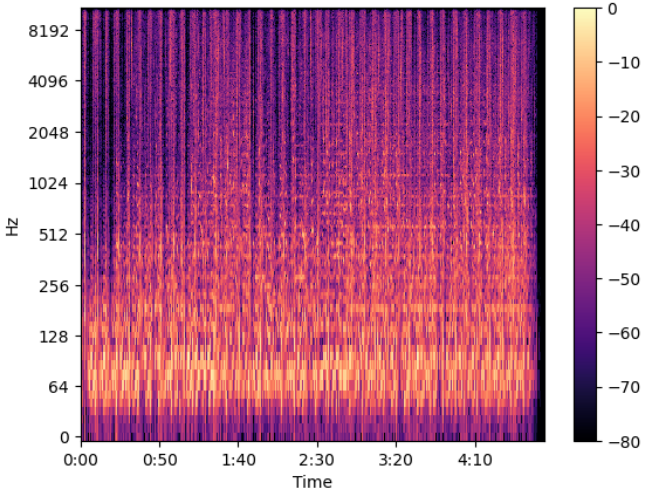


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- fig, ax = plt.pyplot.subplots(): creates a new Matplotlib figure and axes.
  - fig variable holds the entire figure, while
  - ax represents the specific subplot (axes) where the spectrogram will be displayed.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

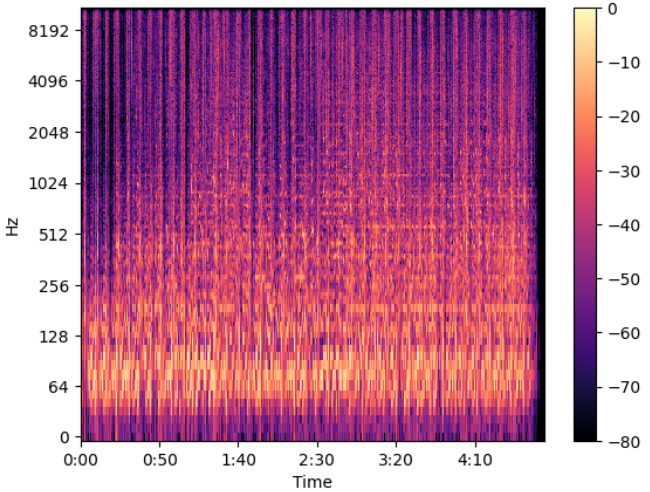


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- img: holds the output of librosa.display.specshow(), which is a handle to the image displayed by Matplotlib.
  - display.specshow() is used to display the spectrogram with various customization options.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

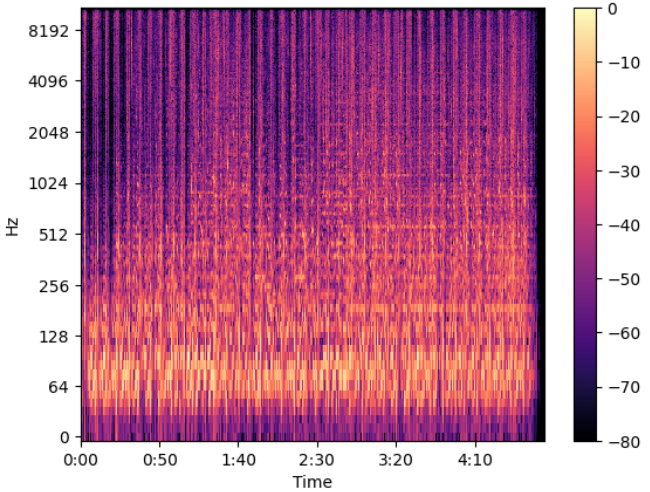


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- img: holds the output of librosa.display.specshow(), which is a handle to the image displayed by Matplotlib.
  - librosa.amplitude_to_db(S_full[:, idx], ref=np.max) converts the magnitude spectrogram slice (S_full[:, idx]) to decibels, scaling relative to the maximum value (ref=np.max) — this makes it easier to interpret, as the decibel scale is more aligned with human hearing.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

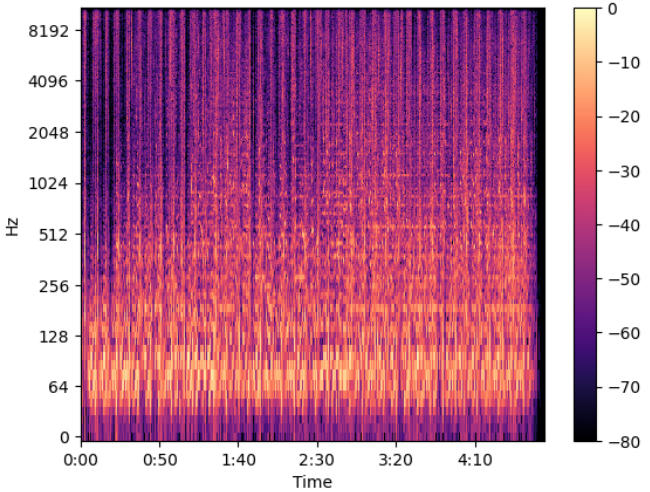


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- Arguments within librosa.display.specshow():
  - y_axis='log': The y-axis is displayed on a logarithmic scale, which is commonly used for spectrograms to give more detail at lower frequencies.
  - x_axis='time': The x-axis is labeled in seconds, representing time.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

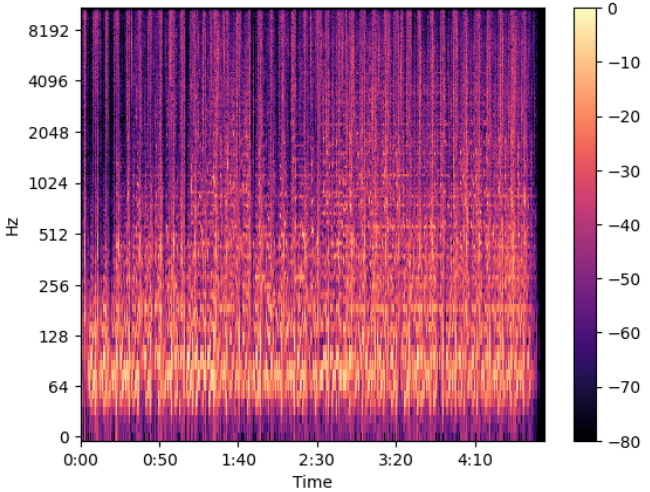


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- Arguments within librosa.display.specshow():
  - sr=sr: This provides the sample rate to Librosa, allowing for accurate time labels on the x-axis.
  - ax=ax: This specifies the axes where the spectrogram should be plotted.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

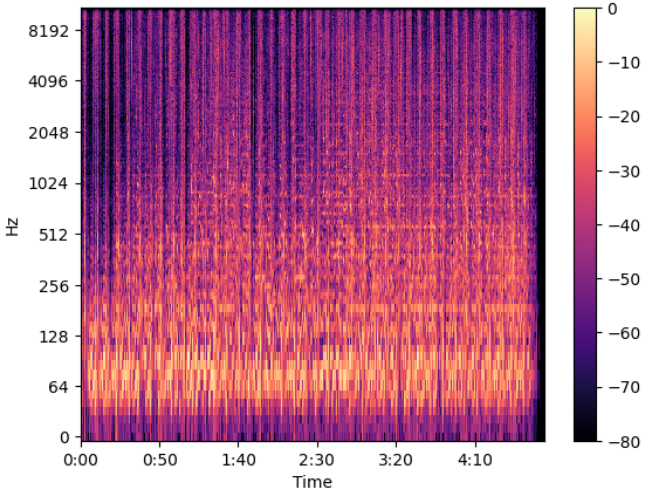


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- fig.colorbar(img, ax=ax): adds a color bar to the spectrogram, mapping colors to amplitude levels in decibels.
  - img is the spectrogram image, and
  - ax=ax specifies that the color bar should be attached to the spectrogram subplot.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```

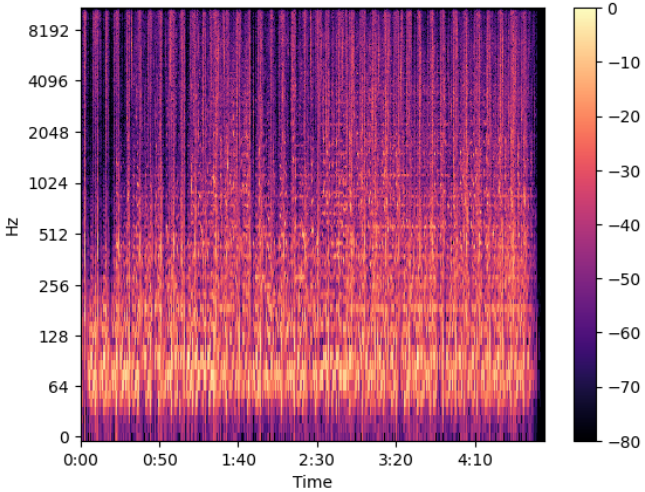


Spectrogram: Song Part Selection

Module III: Sound and Digital Audio

- The resulting plot displays a section of the spectrogram between 90 and 110 seconds, with time on the x-axis, frequency on the y-axis (in log scale), and color representing the amplitude (in dB).
- This gives a visual representation of how the frequencies and their amplitudes change over the selected time window.

```
S_full, phase = librosa.magphase(librosa.stft(y))
idx = slice(*librosa.time_to_frames([90*110], sr=sr))
fig, ax = plt.pyplot.subplots()
img = display.specshow(librosa.amplitude_to_db(S_full[:, idx],
                       ref=np.max), y_axis='log', x_axis='time',
                       sr=sr, ax=ax)
fig.colorbar(img, ax=ax)
```



Spectrogram Visualization

Module III: Sound and Digital Audio

- Using Librosa and Matplotlib.

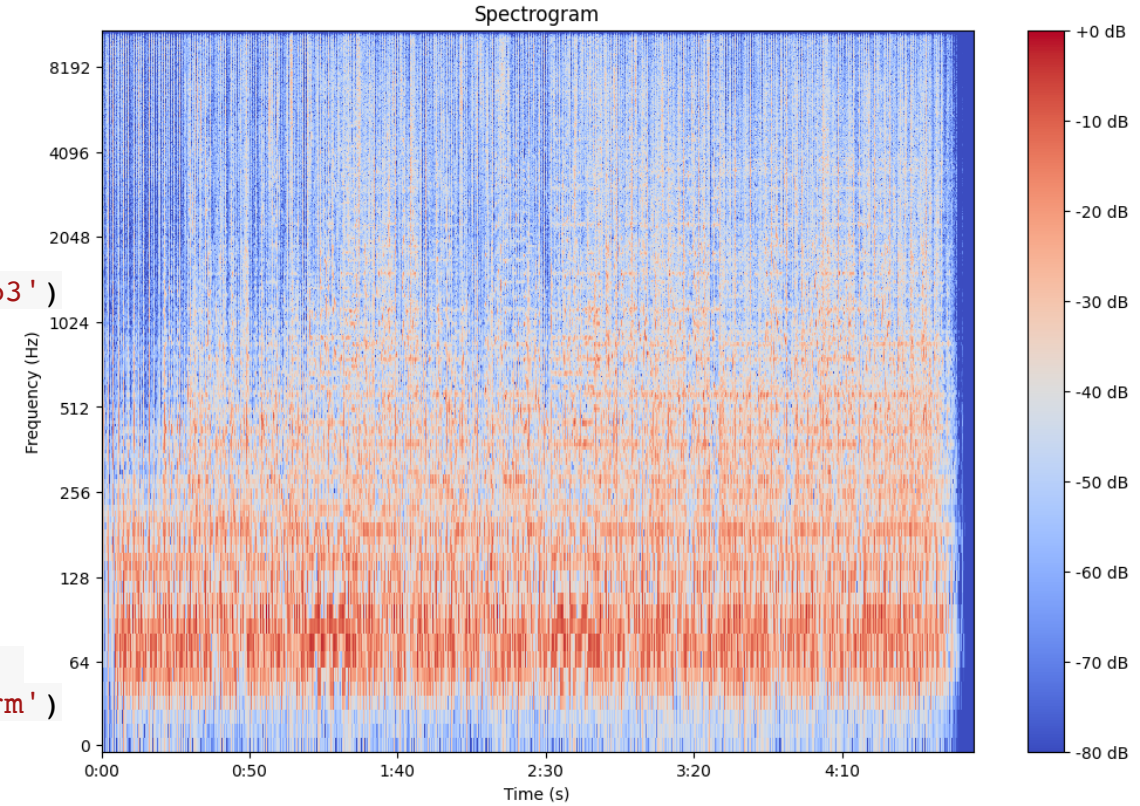
```
import … # all in previous page
import librosa.display

# Load the audio file
y, sr = librosa.load('/mydrive/Set01/01BillieJean.mp3')

# Compute the STFT to get the spectrogram
D = np.abs(librosa.stft(y))

# Convert the amplitude spectrogram to a dB scale
D_dB = librosa.amplitude_to_db(D, ref=np.max)

# Plot the spectrogram
plt.figure(figsize=(12, 8))
librosa.display.specshow(D_dB, sr=sr, x_axis='time',
                         y_axis='log', cmap='coolwarm')
plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.show()
```



Digital Audio Practice

Vocal Extraction



Vocal Extraction: Demucs

Module III: Sound and Digital Audio

- Demucs (DEep Music Source Separation) is an open-source deep learning-based tool for separating music into its constituent stems (vocals, drums, bass, etc.).
- It was developed by Facebook AI Research (FAIR) and is considered one of the state-of-the-art music source separation tools.

Vocal Extraction: Demucs

Module III: Sound and Digital Audio

- Key Features: Advanced Architecture
  - Uses a U-Net-like architecture with a convolutional encoder-decoder structure
  - Incorporates LSTM (Long Short-Term Memory) layers for temporal modeling
  - Employs attention mechanisms for better separation quality

Vocal Extraction: Demucs

Module III: Sound and Digital Audio

- Key Features: Pre-trained Models
  - Provide the models for music analysis purposes

```
from demucs.pretrained import get_model

# Available models
model = get_model('htdemucs')          # Default 4-stem model (recommended)
model = get_model('htdemucs_6s')       # 6-stem model
model = get_model('demucs')            # Original Demucs model
model = get_model('demucs_extra')      # Extra trained model
model = get_model('demucs_quantized')# Quantized (smaller) model
```

Vocal Extraction: Demucs

Module III: Sound and Digital Audio

- Key Features: Installation
  - Using

```
!pip install demucs
```

Vocal Extraction: Demucs

Module III: Sound and Digital Audio

- Key Features: Examples
  - Separate vocal and instrument sounds

```
# Install demucs
!pip install demucs

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

import os

# Define paths
input_audio = '/content/drive/MyDrive/Sounds-of-Music/Songs/02Bitch.mp3'
output_directory = '/content/drive/MyDrive/Sounds-of-Music/Outputs/'

# Create output directory
os.makedirs(output_directory, exist_ok=True)

# Use demucs via command line (most reliable method)
!demucs --two-stems=vocals "{input_audio}" -o "{output_directory}"

print("Separation complete!")
```

Spectrogram Visualization

Module III: Sound and Digital Audio

- Librosa is a popular Python library used primarily for audio and music analysis.
- It provides tools that make it easy to work with audio data:
- Visualization: the “librosa” can be combined with Matplotlib to visualize audio data, such as waveforms, spectrograms, and other features, making it easier to analyze and understand the structure of the audio.

```
!pip install librosa matplotlib IPython
```

Spectrogram Visualization

Module III: Sound and Digital Audio

- Using Librosa and Matplotlib.

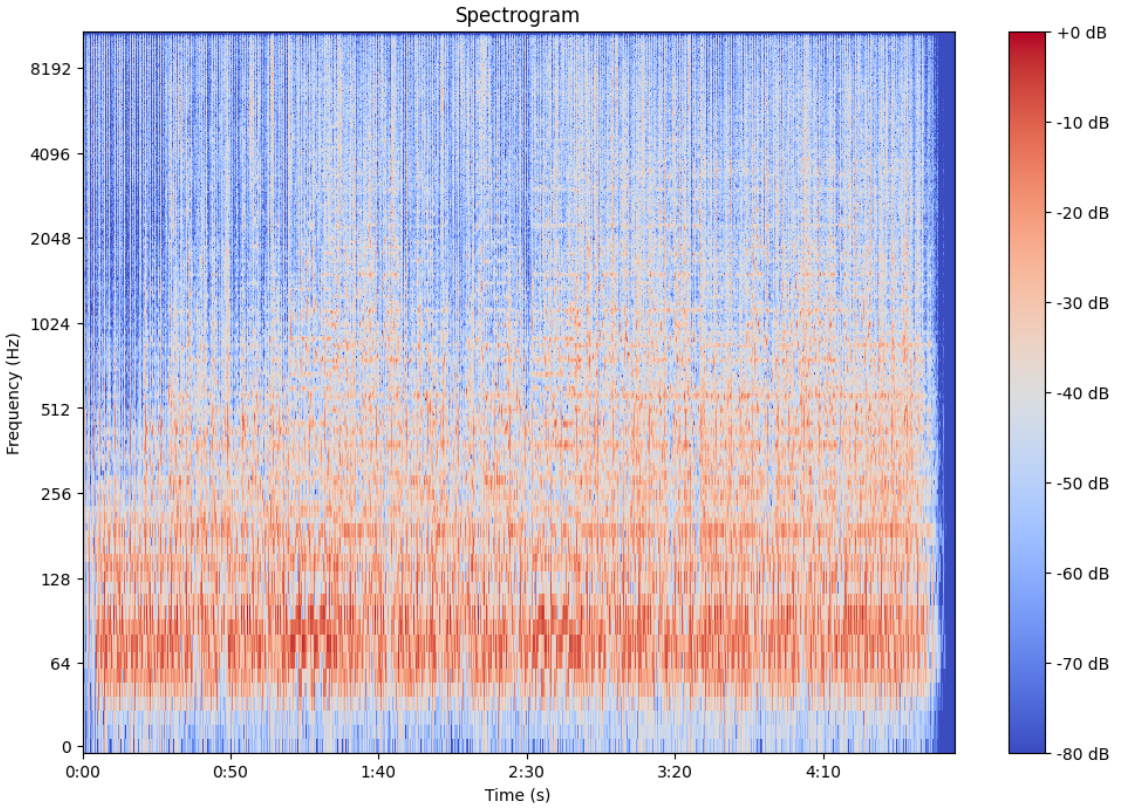
```
import librosa
import numpy as np
import matplotlib.pyplot as plt
import librosa.display

# Load the audio file
y, sr = librosa.load('/mydrive/Set01/02Bitch.mp3')

# Compute the STFT to get the spectrogram
D = np.abs(librosa.stft(y))

# Convert the amplitude spectrogram to a decibel (dB) scale
D_dB = librosa.amplitude_to_db(D, ref=np.max)

# Plot the spectrogram
plt.figure(figsize=(12, 8))
librosa.display.specshow(D_dB, sr=sr, x_axis='time',
                         y_axis='log', cmap='coolwarm')
plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.show()
```



Spectrogram: spleeter Extraction Output

Module III: Sound and Digital Audio

- Using Librosa and Matplotlib.

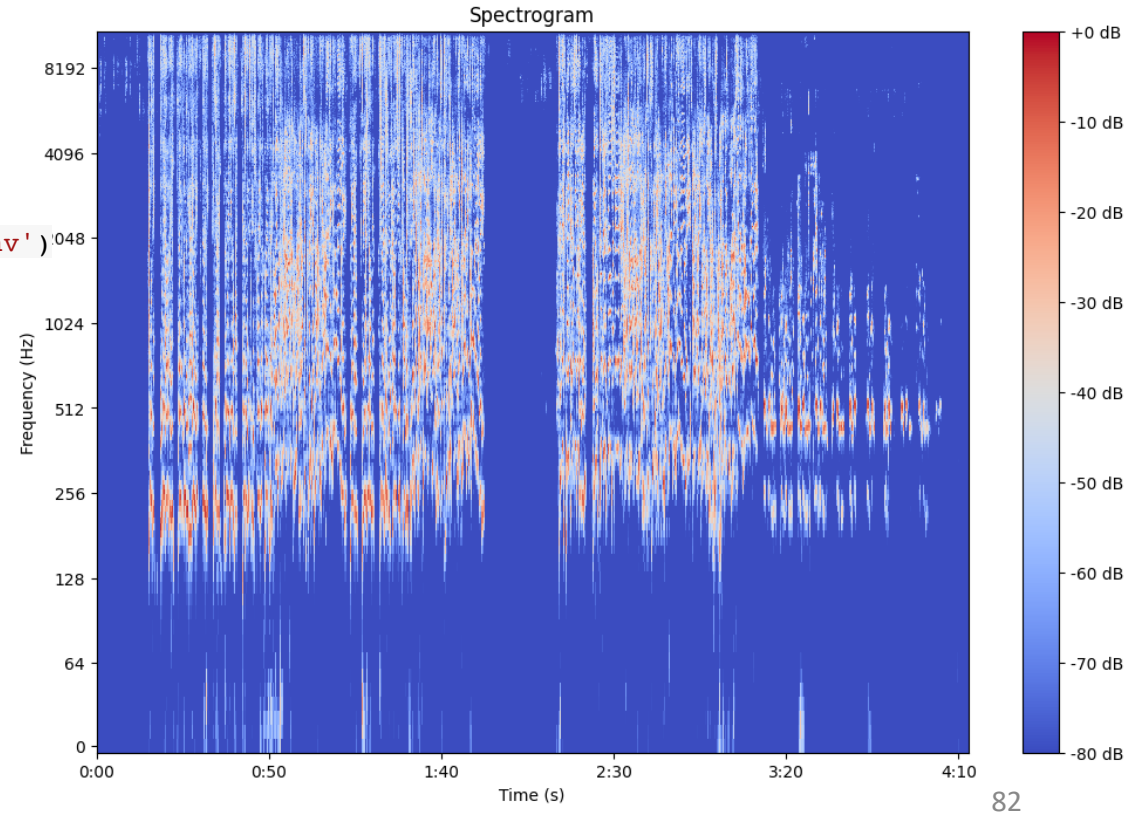
```
import librosa
import numpy as np
import matplotlib.pyplot as plt
import librosa.display

# Load the audio file
v, sr = librosa.load('/mydrive/Set01/output/02Bitch/vocals.wav')

# Compute the STFT to get the spectrogram
D = np.abs(librosa.stft(v))

# Convert the amplitude spectrogram to a decibel (dB) scale
D_dB = librosa.amplitude_to_db(D, ref=np.max)

# Plot the spectrogram
plt.figure(figsize=(12, 8))
librosa.display.specshow(D_dB, sr=sr, x_axis='time',
                         y_axis='log', cmap='coolwarm')
plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.show()
```



Spectrogram: spleeter Extraction Output

Module III: Sound and Digital Audio

- Using Librosa and Matplotlib.

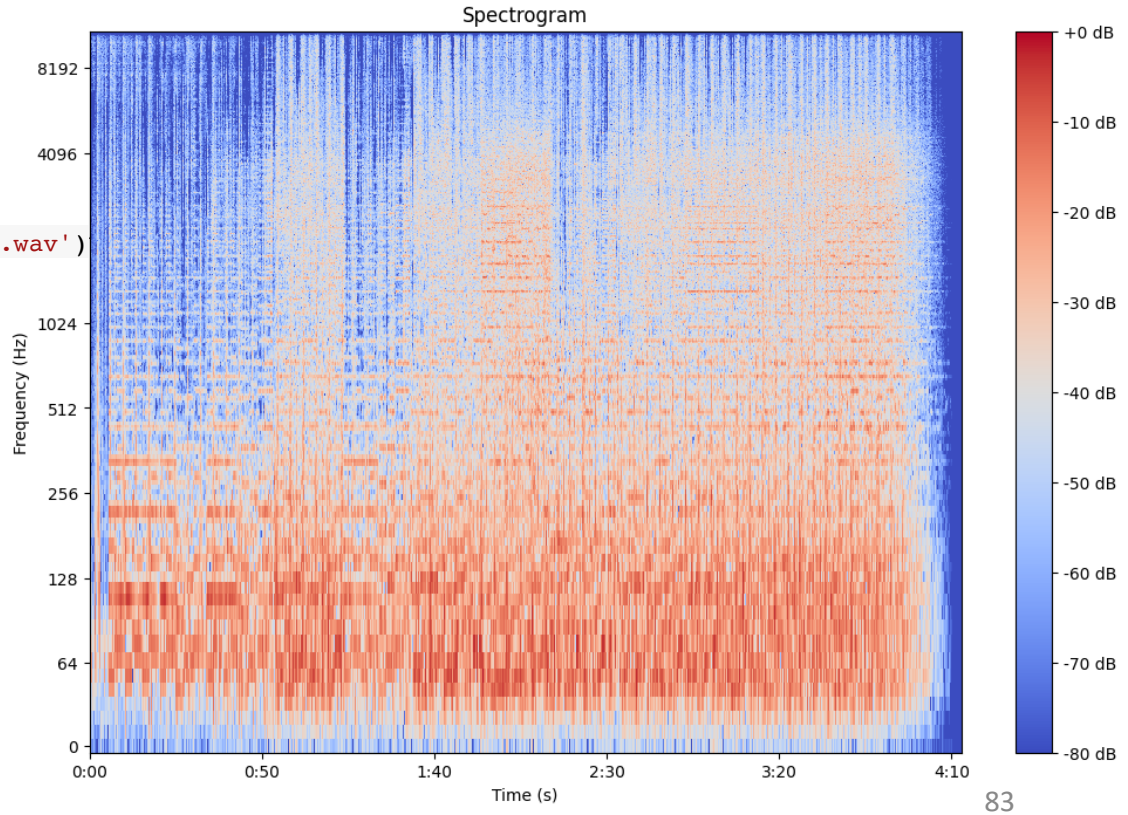
```
import librosa
import numpy as np
import matplotlib.pyplot as plt
import librosa.display

# Load the audio file
a, sr = librosa.load('/mydrive/Set01/output/02Bitch/accompaniment.wav')

# Compute the STFT to get the spectrogram
D = np.abs(librosa.stft(a))

# Convert the amplitude spectrogram to a decibel (dB) scale
D_dB = librosa.amplitude_to_db(D, ref=np.max)

# Plot the spectrogram
plt.figure(figsize=(12, 8))
librosa.display.specshow(D_dB, sr=sr, x_axis='time',
                         y_axis='log', cmap='coolwarm')
plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.show()
```



Spectrogram: spleeter Extraction Output

Module III: Sound and Digital Audio

- Using Librosa and Matplotlib.

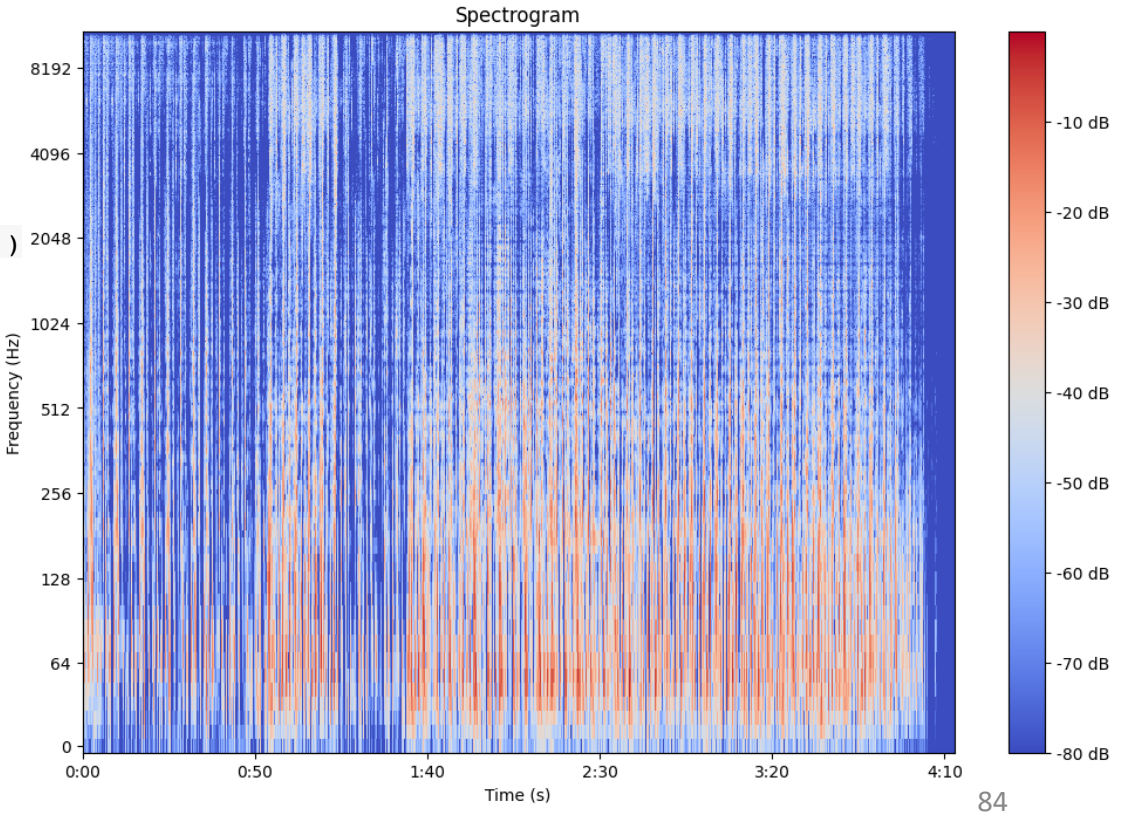
```
import librosa
import numpy as np
import matplotlib.pyplot as plt
import librosa.display

# Load the audio file
d, sr = librosa.load('/mydrive/Set01/output/02Bitch/drums.wav')

# Compute the STFT to get the spectrogram
D = np.abs(librosa.stft(d))

# Convert the amplitude spectrogram to a decibel (dB) scale
D_dB = librosa.amplitude_to_db(D, ref=np.max)

# Plot the spectrogram
plt.figure(figsize=(12, 8))
librosa.display.specshow(D_dB, sr=sr, x_axis='time',
                         y_axis='log', cmap='coolwarm')
plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.show()
```



Spectrogram: spleeter Extraction Output

Module III: Sound and Digital Audio

- Using Librosa and Matplotlib.

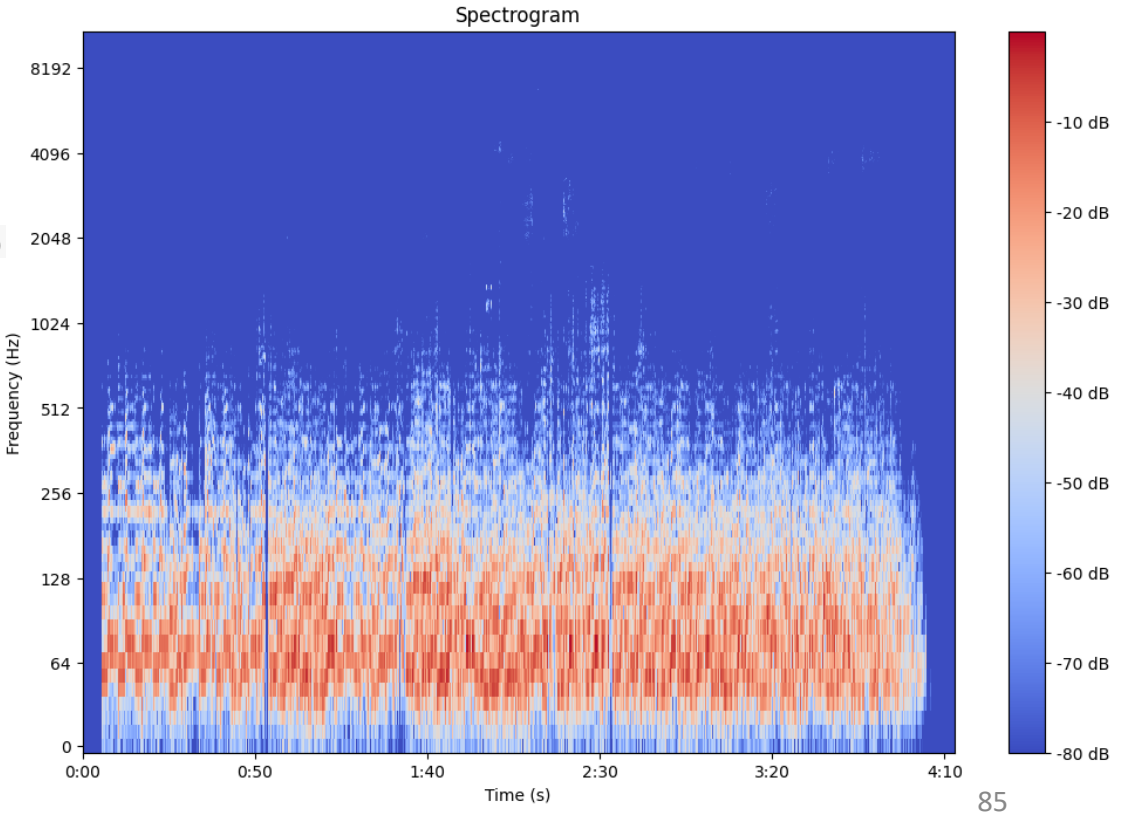
```
import librosa
import numpy as np
import matplotlib.pyplot as plt
import librosa.display

# Load the audio file
b, sr = librosa.load('/mydrive/Set01/output/02Bitch/bass.wav')

# Compute the STFT to get the spectrogram
D = np.abs(librosa.stft(b))

# Convert the amplitude spectrogram to a decibel (dB) scale
D_dB = librosa.amplitude_to_db(D, ref=np.max)

# Plot the spectrogram
plt.figure(figsize=(12, 8))
librosa.display.specshow(D_dB, sr=sr, x_axis='time',
                         y_axis='log', cmap='coolwarm')
plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.show()
```



Spectrogram: spleeter Extraction Output

Module III: Sound and Digital Audio

- Using Librosa and Matplotlib.

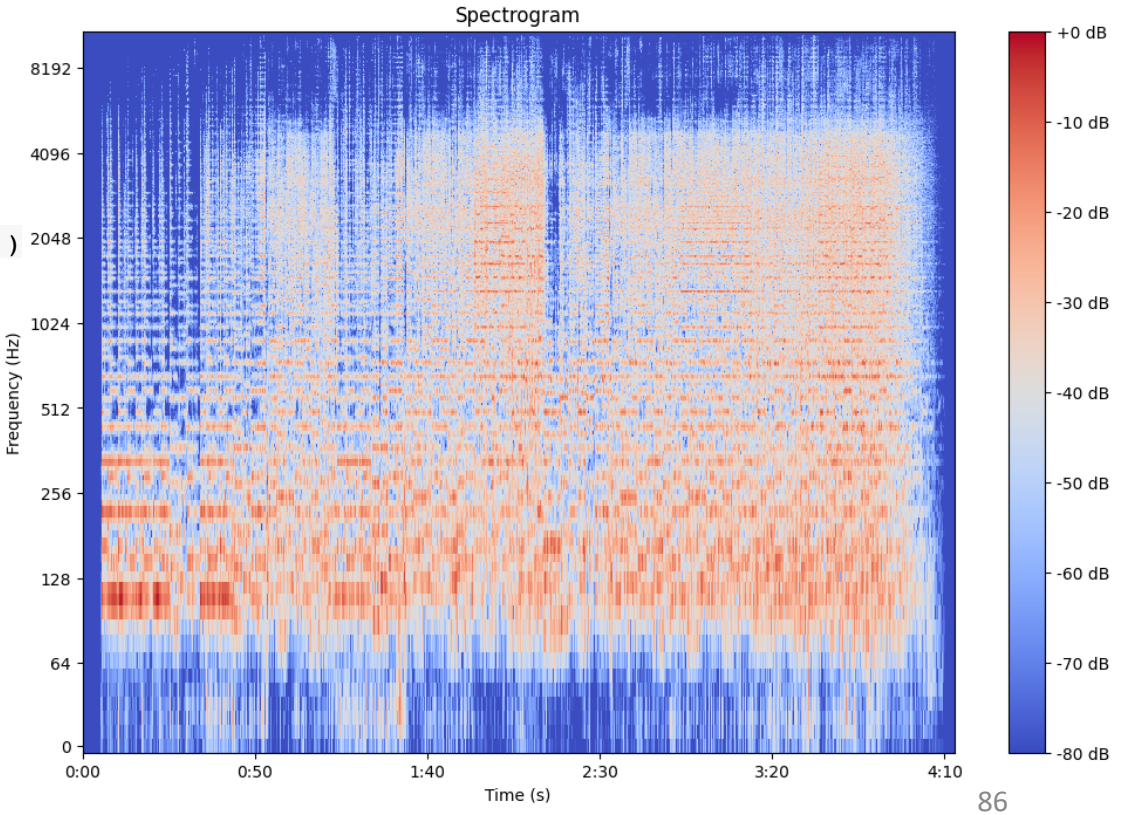
```
import librosa
import numpy as np
import matplotlib.pyplot as plt
import librosa.display

# Load the audio file
o, sr = librosa.load('/mydrive/Set01/output/02Bitch/other.wav')

# Compute the STFT to get the spectrogram
D = np.abs(librosa.stft(o))

# Convert the amplitude spectrogram to a decibel (dB) scale
D_dB = librosa.amplitude_to_db(D, ref=np.max)

# Plot the spectrogram
plt.figure(figsize=(12, 8))
librosa.display.specshow(D_dB, sr=sr, x_axis='time',
                         y_axis='log', cmap='coolwarm')
plt.colorbar(format='%+2.0f dB')
plt.title('Spectrogram')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.show()
```



Spectrogram: demucs Extraction Output

Module III: Sound and Digital Audio

- Challenge:
  - How to extract parts of music using demucs?

Sound Emotion Recognition (SER)

Preparing Environment

- `Library Installation`

```
# @title Environment Setup (one time run)
!pip install -q librosa soundfile matplotlib numpy scikit-learn tensorflow seaborn ipywidgets streamlit pyngrok pandas
!pip install -q demucs  # for music sound extraction
```

librosa

- Audio management: Load files, extract features (MFCC, Mel-spectrogram), display waveform.
- Example functions/usages:
  - librosa.load()
  - librosa.feature.mfcc()
  - librosa.display.waveshow()

soundfile

- Read/write .wav audio files (can be used as a replacement for librosa, but smaller).
- Example functions/usages:
  - sf.read(), sf.write() – Used when recording processed audio.

matplotlib

- Draw all graphs: waveform, spectrogram, confusion matrix
- Example functions/usages:
  - plt.plot(), plt.imshow(), librosa.display.specshow() – perform well with matplotlib

numpy

- Manages arrays of audio signals and performs mathematical calculations (mean, std, FFT).
- Example functions/usages:
  - np.mean(), np.sin() (creates a test waveform), np.fft.fft()

scikit-learn

- Applying Machine Learning to create SER model, preparing train/test, scale features, encode labels
- Example functions/usages:
  - train_test_split(), StandardScaler(), RandomForestClassifier(), LabelEncoder()

tensorflow

- Applying Deep Learning models (CNN, LSTM) for training spectrogram
- Example functions/usages:
  - tf.keras.Sequential, Conv2D, model.fit()

seaborn

- Make more beautiful visualization, using plot confusion matrix
- Example functions/usages:
  - sns.heatmap() – To display the model's prediction results.

ipywidgets

- Create interactive UI in Colab (buttons, file upload, output)
- Example functions/usages:
  - widgets.Button(), widgets.FileUpload(), widgets.Output()

streamlit

- Create a web app to demonstrate the model.
- Example functions/usages:
  - st.title(), st.file_uploader(), st.pyplot() – to create dashboard

pyngrok

- Open/create a public URL for Streamlit that can run on Colab
- Example functions/usages:
  - ngrok.connect(8501) – Allows outsiders to access the web app.

pandas

- Organize the data into a table (DataFrame) for the feature dataset.
- Example functions/usages:
  - pd.DataFrame(), df.groupby().mean() – Used for creating and analyzing data sets of features

demucs

- Separating vocals from background music — used in the topic "Vocal Extraction" — allows you to analyze the emotion of spoken words without being affected by the music.
- Example functions/usages:
  - `!demucs --two-stems=vocals "audio.mp3" -o output/`
  - This will produce vocals.wav (vocals) and no_vocals.wav (background music) files, improving the accuracy of emotional detection in speech amid music.

Preparing Environment

- Run this cell at startup to enable the library and set basic parameters.

```
import librosa
import librosa.display
import soundfile as sf
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import pandas as pd
import seaborn as sns
from IPython.display import Audio, display
import warnings
warnings.filterwarnings('ignore')


print("Now available!")
```

Load Audio and Display Waveform.

- Download sample files from librosa

```
# Download sample files
audio_path = librosa.example('trumpet')
y, sr = librosa.load(audio_path, sr=None)

print(f"Sample rate: {sr} Hz")
print(f"Duration: {len(y)/sr:.2f} seconds")
print(f"Number of samples: {len(y)}")

# Show Waveform
plt.figure(figsize=(14, 4))
librosa.display.waveshow(y, sr=sr, alpha=0.7, color='blue')
plt.title('Waveform')
plt.xlabel('Time (second)')
plt.ylabel('‘Amplitude')
plt.grid(alpha=0.3)
plt.show()
```

Load Audio and Display Waveform.

- `Play audio`

```
# play audio
display(Audio(y, rate=sr))
```

Key Name	Description	Sound of …	Example
'trumpet'	Mihai Sorohan - Trumpet loop	trumpet	librosa.example('trumpet')

Sound Samples in LibROSA.

- LibROSA provides many sample sounds to try out.
- Besides 'trumpet,' we are particularly interested in the speech sound, which is good for testing emotion recognition.

Key Name	Description	Sound of ...	Example
'brahms'	Brahms - Hungarian Dance #5	Classical music, for testing tempo.	librosa.example('brahms')
'choice'	Admiral Bob - Choice	Short electronic music snippets are suitable for isolating sound sources.	librosa.example('choice')
'fishin'	Karissa Hobbs - Let's Go Fishin'	Folk/pop music with vocals.	librosa.example('fishin')
'humpback'	Humpback whale song	Whale sounds / Sounds from nature	librosa.example('humpback')

Sound Samples in LibROSA.

- More examples like 'libri1', 'libri2', 'libri3' are human voices suitable for SER applications.

Key Name	คำอธิบาย (EN)	ภาษาไทย / ลักษณะเสียง	ตัวอย่างการใช้งาน
'libri1'	Ashiel Mystery (LibriSpeech)	หนังสือเสียง, เป็นเรื่องเล่า (เสียงผู้ชาย)	librosa.example('libri1')
'libri2'	The Age of Chivalry (LibriSpeech)	หนังสือเสียง, เป็นเรื่องเล่า (เสียงผู้ชาย)	librosa.example('libri2')
'libri3'	Sense and Sensibility (LibriSpeech)	หนังสือเสียง, เป็นเรื่องเล่า (เสียงผู้หญิง)	librosa.example('libri3')
'nutcracker'	Tchaikovsky - Dance of the Sugar Plum Fairy	เพลงออเครสตร้า	librosa.example('nutcracker')

Sound Samples in LibROSA.

- And more here

Key Name	คำอธิบาย (EN)	ภาษาไทย / ลักษณะเสียง	ตัวอย่างการใช้งาน
'pistachio'	The Piano Lady - Pistachio Ice Cream	เพลงเปียโนแร็กไทม์	librosa.example('pistachio')
'robin'	Bird Whistling, Robin	เสียงนกหวูด	librosa.example('robin')
'sweetwaltz'	Setuniman - Sweet Waltz	เพลงวอลทซ์	librosa.example('sweetwaltz')
'vibeace'	Kevin Macleod - Vibe Ace	เสียงไวบราโฟน เบส และเปียโน	librosa.example('vibeace')

Spectrogram
vs.
Mel-Spectrogram

Spectrogram vs. Mel-Spectrogram

- Calculate STFT and convert to dB scale.
- Create Mel-Spectrogram.
  - Mel stands for Melody.
  - Mel scale is a frequency scale that mimics the human ear.
  - n_mels is the number of rows (resolution) in the Mel-spectrogram.
  - Recommended values: 128 (default), 64 (for faster/softer playback), 256 (high resolution).

```
D = np.abs(librosa.stft(y))
D_db = librosa.amplitude_to_db(D, ref=np.max)

# Create Mel-Spectrogram
mel_spec = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)
mel_db = librosa.power_to_db(mel_spec, ref=np.max)
```

Spectrogram vs. Mel-Spectrogram

- D_db is the spectrogram converted to dB.
- sr=sr is the sample rate used to adjust the time and frequency axes to be correct.
- x_axis='time': Let the x-axis be time (seconds).
- y_axis='log' for the spectrogram: Let the y-axis be the logarithmic frequency (Hz) (not mel).

```
# Show a comparison image.
fig, ax = plt.subplots(1, 2, figsize=(15, 5))

img1 = librosa.display.specshow(D_db, sr=sr, x_axis='time', y_axis='log', ax=ax[0])
ax[0].set_title('Spectrogram (Logarithmic frequency)')
fig.colorbar(img1, ax=ax[0], format='%+2.0f dB')

img2 = librosa.display.specshow(mel_db, sr=sr, x_axis='time', y_axis='mel', ax=ax[1])
ax[1].set_title('Mel-Spectrogram')
fig.colorbar(img2, ax=ax[1], format='%+2.0f dB')

plt.tight_layout()
plt.show()
```

Spectrogram vs. Mel-Spectrogram

- mel_db is the mel-spectrogram converted to dB.
- sr=sr is the sample rate used to adjust the time and frequency axis scales correctly.
- y_axis= 'mel' for mel-spectrogram: set the y-axis to mel scale.
- ax=ax[0] or ax=ax[1] specify the axes to plot.

```
# Show a comparison image.
fig, ax = plt.subplots(1, 2, figsize=(15, 5))

img1 = librosa.display.specshow(D_db, sr=sr, x_axis='time', y_axis='log', ax=ax[0])
ax[0].set_title('Spectrogram (Logarithmic frequency)')
fig.colorbar(img1, ax=ax[0], format='%+2.0f dB')

img2 = librosa.display.specshow(mel_db, sr=sr, x_axis='time', y_axis='mel', ax=ax[1])
ax[1].set_title('Mel-Spectrogram')
fig.colorbar(img2, ax=ax[1], format='%+2.0f dB')

plt.tight_layout()
plt.show()
```

Spectrogram vs. Mel-Spectrogram

- `fig.colorbar()` adds a color bar displaying the dB scale.
- `format='%+0.2f dB'`: Color bar numeric format (0 decimal places, with +/– signs, followed by dB)

```
# Show a comparison image.
fig, ax = plt.subplots(1, 2, figsize=(15, 5))

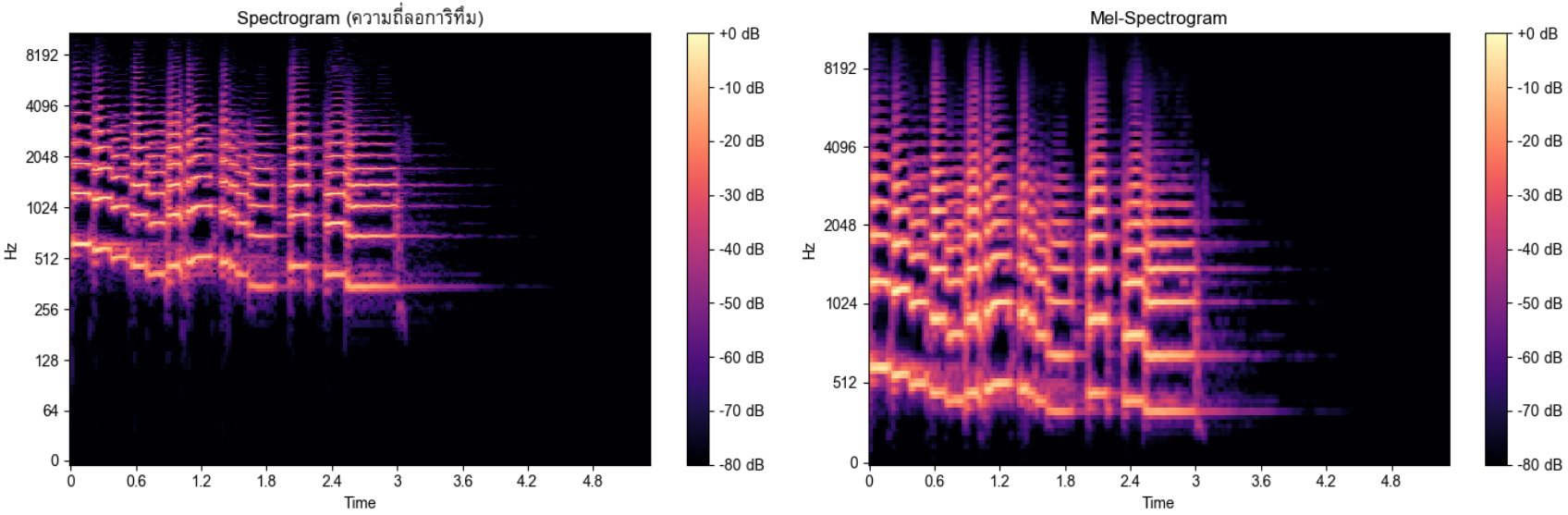
img1 = librosa.display.specshow(D_db, sr=sr, x_axis='time', y_axis='log', ax=ax[0])
ax[0].set_title('Spectrogram (Logarithmic frequency)')
fig.colorbar(img1, ax=ax[0], format='%+2.0f dB')

img2 = librosa.display.specshow(mel_db, sr=sr, x_axis='time', y_axis='mel', ax=ax[1])
ax[1].set_title('Mel-Spectrogram')
fig.colorbar(img2, ax=ax[1], format='%+2.0f dB')

plt.tight_layout()
plt.show()
```

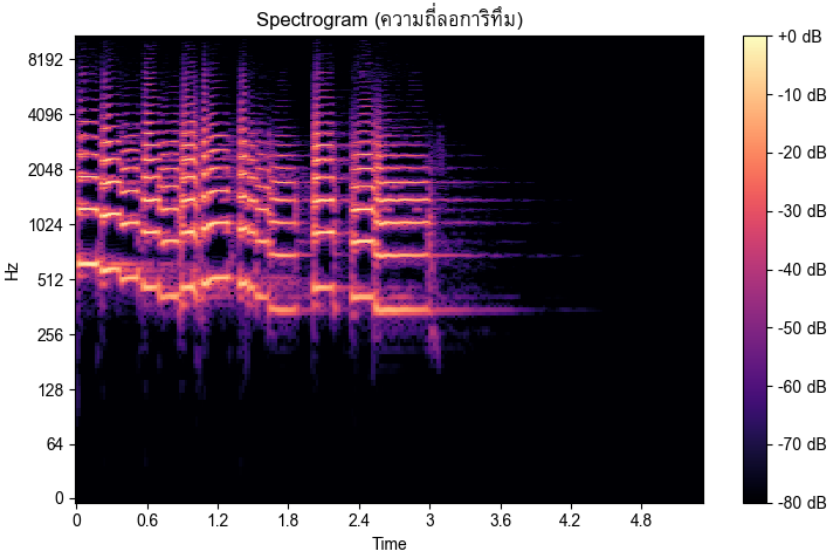
Spectrogram vs. Mel-Spectrogram

- Figures



Spectrogram

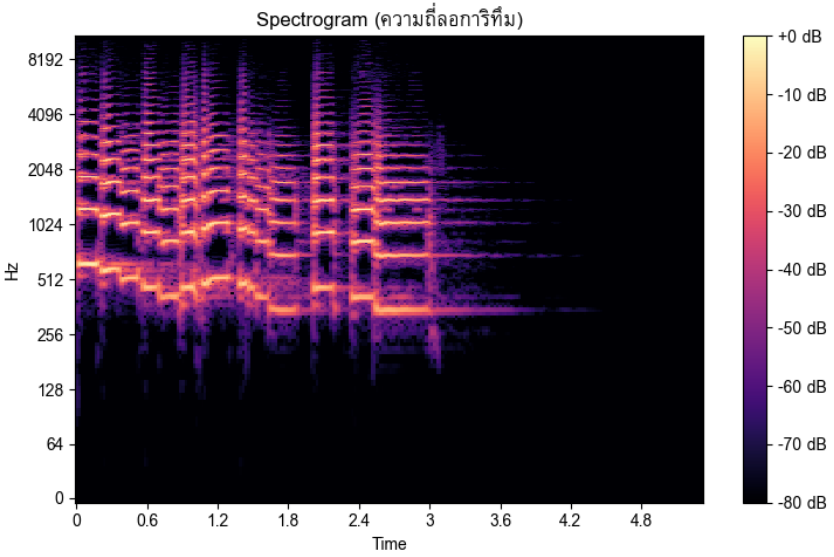
- Figure of Spectrogram



- What is a spectrogram?
- A spectrogram is a 3D representation of an audio signal.
  - Horizontal axis (X): Time (seconds)
  - Vertical axis (Y): Frequency (Hz)
  - Brightness/Color: Sound power or intensity (dB)

Spectrogram

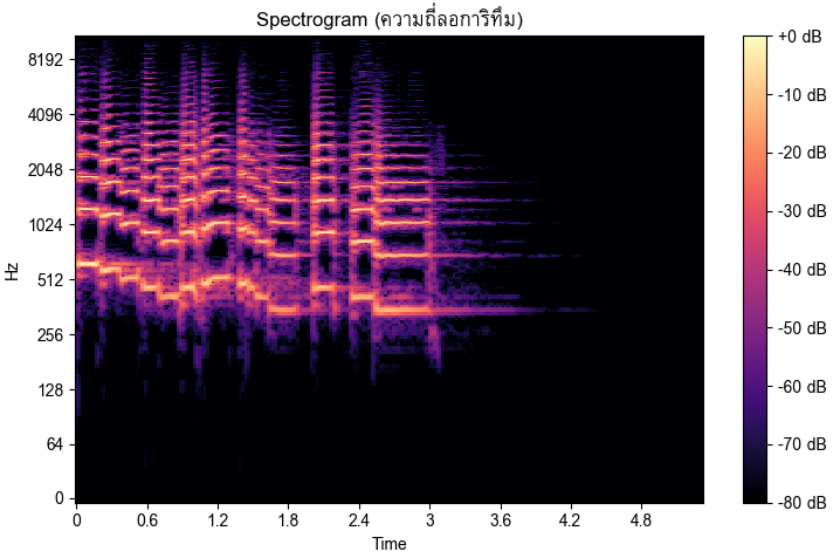
- Figure of Spectrogram



- How it's created:
  - Sound is divided into short frames (e.g., 25 milliseconds per frame).
  - Each frame is then transformed using the Short-Time Fourier Transform (STFT) to determine its frequency and energy.
  - These frames are then stitched together to create an image.

Spectrogram

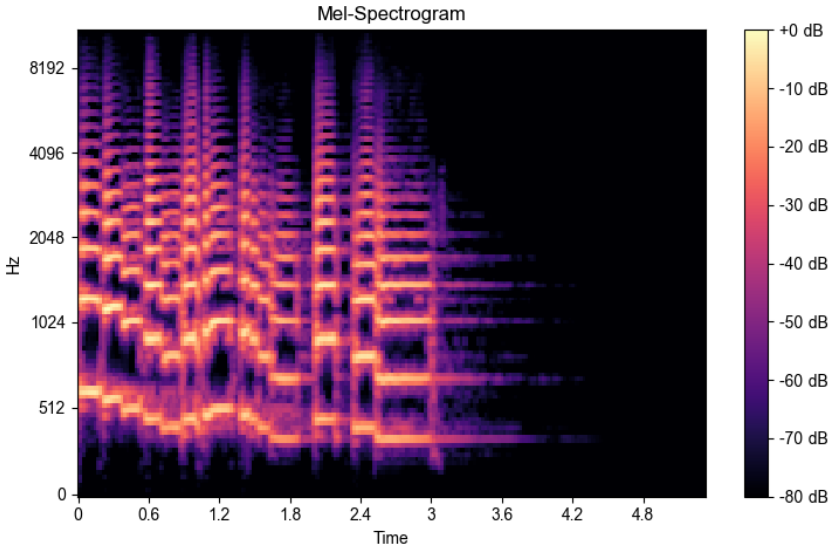
- Figure of Spectrogram



- Features:
  - The frequency axis is linear – the distance between 100 Hz and 200 Hz is equal to the distance between 1000 Hz and 1100 Hz.
  - Provides complete high-frequency detail.
  - Example image: Speech waveforms show vertical bars (each frame) and vertical frequency channels (different frequencies).

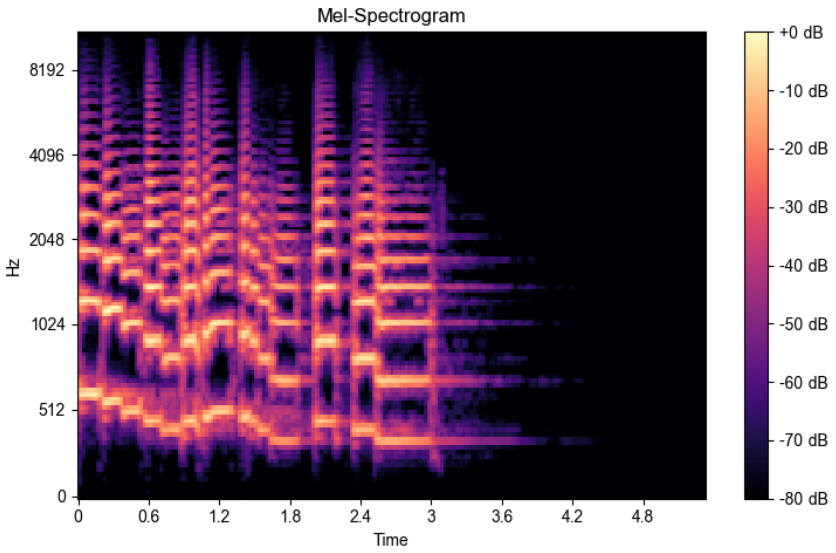
Mel-Spectrogram

- Figure of Mel-Spectrogram
- What is a Mel-Spectrogram?
- A Mel-Spectrogram is a spectrogram where the frequency axis is converted to the Mel scale – a scale that mimics the hearing of the human ear.



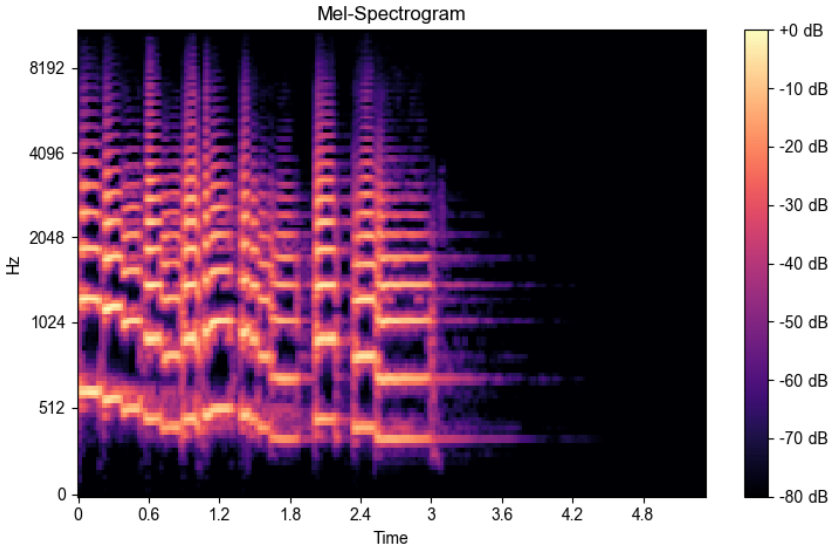
Mel-Spectrogram

- The parameter n_mels refers to the number of rows (bins) of the Mel filter bank to be used in converting a linear-frequency spectrogram to a Mel-spectrogram.
- This value represents the “resolution” on the Mel axis – increasing it (e.g., 256) captures more frequency detail but increases the image size.
- For SER applications, 128 is commonly used as it is sufficient for extracting formants and timbre.



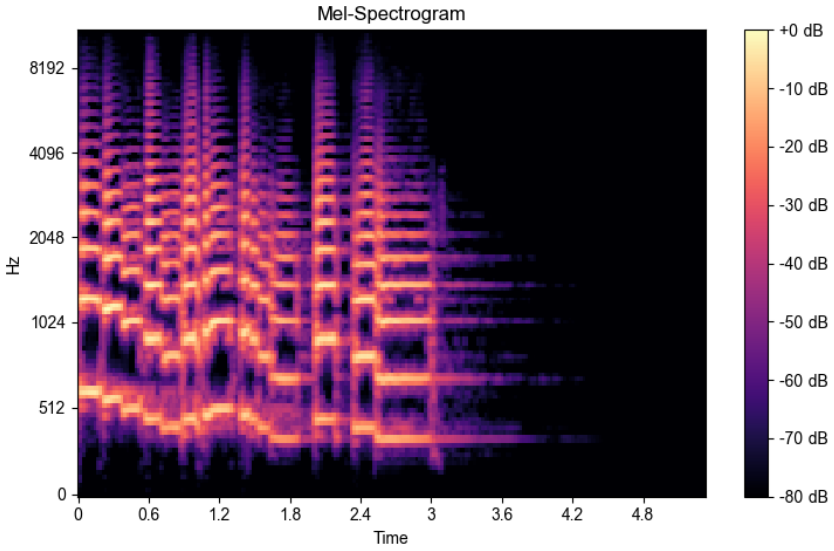
Mel-Spectrogram

- How the Mel scale works:
  - The human ear distinguishes low frequencies (e.g., 100–500 Hz) more precisely than high frequencies (e.g., 5000–8000 Hz).
  - The Mel scale causes the distance between frequencies to be unequal: at low frequencies, they are “stretched,” making the difference more apparent; at high frequencies, they are “contracted.”



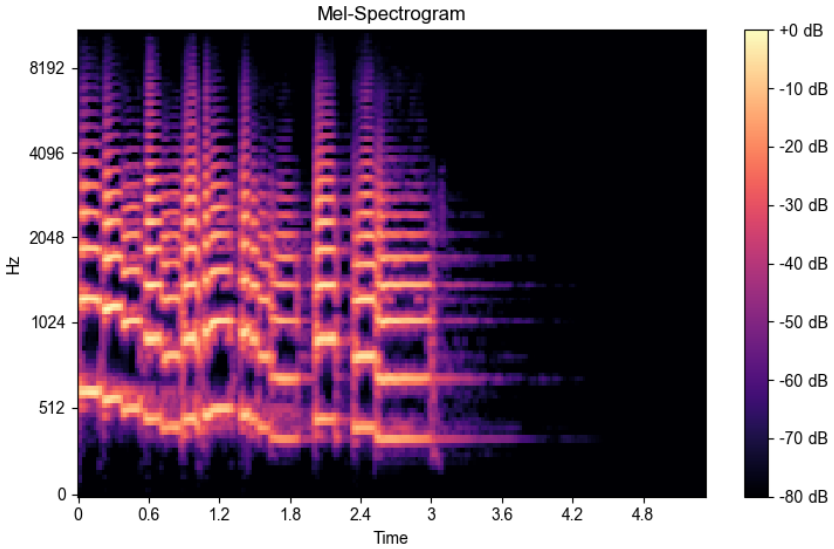
Mel-Spectrogram

- How to create:
  - Multiply the spectrogram by the Mel filter banks (groups of triangular filters arranged according to the Mel scale) and then combine the energies in the same frequency range.



Mel-Spectrogram

- Properties:
  - Reduces the size of the frequency information (e.g., from 513 frequency points to 128 Mel bands).
  - Emphasizes the characteristics that the human ear uses to distinguish sounds (formants, timbre).

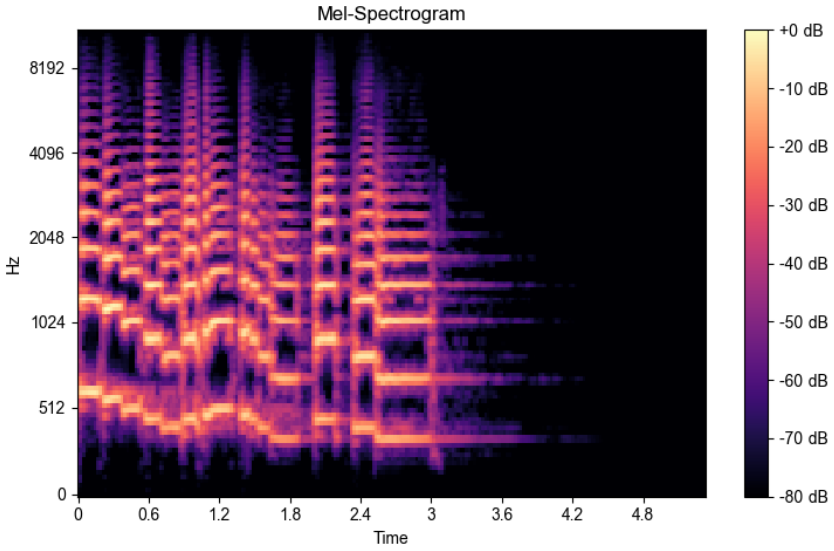


Formants vs. Timbre

Different sound patterns and sound characteristics.

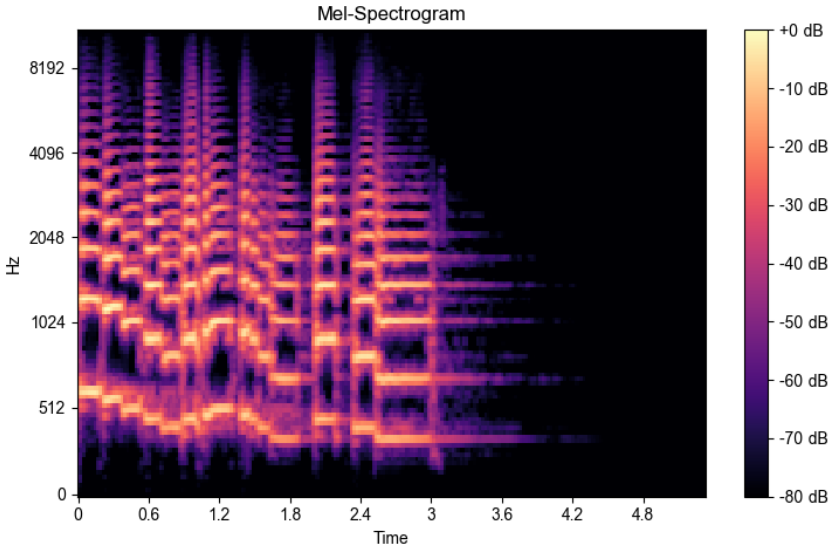
Formants

- Formants are the frequency ranges that are most amplified when we pronounce words, caused by the resonance of the vocal cords (throat, mouth, nasal cavity).



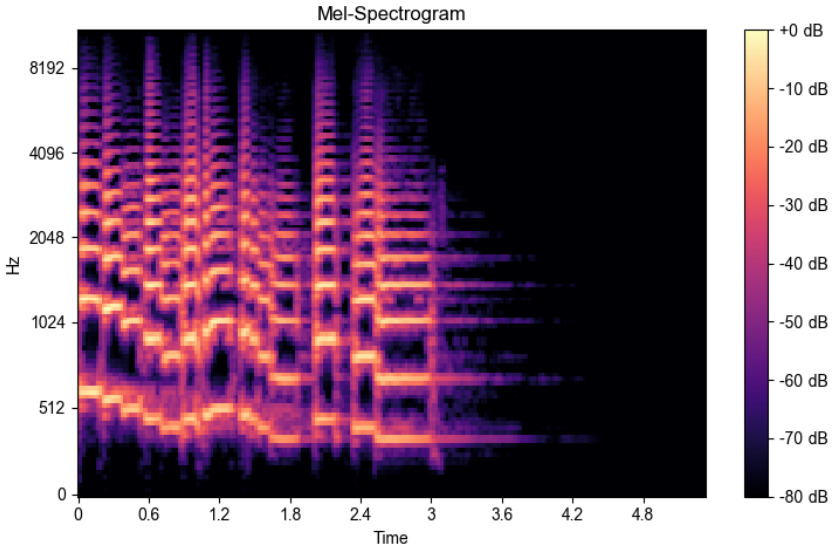
Formants

- Each vowel sound (a, e, i, o, u) has a specific formant position (called F1, F2, F3…).
- F1 is related to the height of the tongue (the sound 'oo' vs 'ee').
- F2 is related to the forward–backward position of the tongue (the sound 'ee' vs 'o').



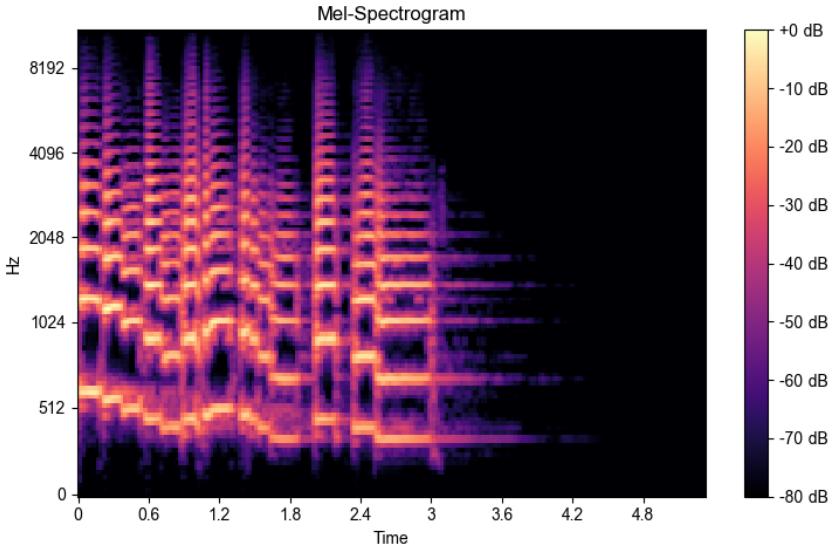
Formants

- Formants in the spectrogram appear as dark horizontal bands at specific frequencies, such as F1 at 300–800 Hz and F2 at 800–2500 Hz.
- Connected to Emotion:
- When we are excited or angry, the muscles around the vocal cords tense → formants shift higher, changing the vowel sound.
- When we are sad, the voice becomes lower → formants drop.



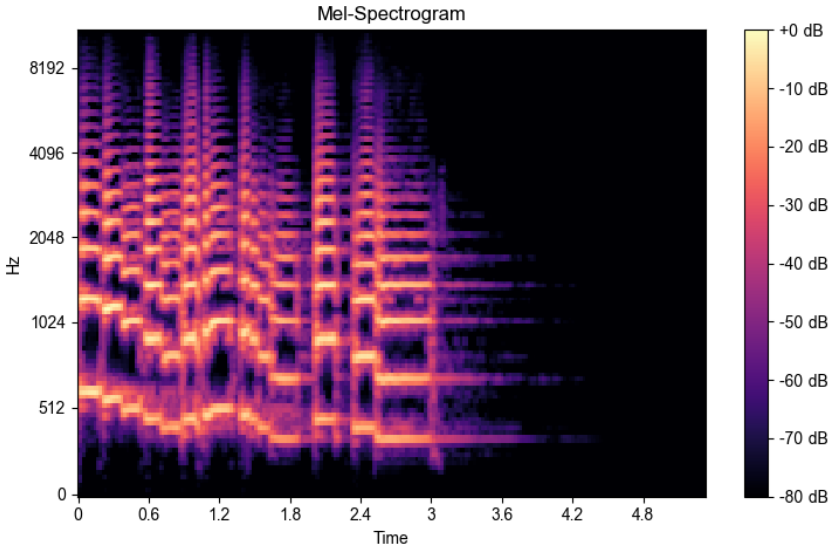
Timbre

- Timbre refers to the color or quality of a sound that allows us to distinguish whose voice it is or what type of instrument it is, even when singing the same note (same pitch).
- It is created by harmonics (overtones) that are above the fundamental frequency.
- Each person has a different timbre depending on their physiology (neck size, mouth shape, vocal cord characteristics).



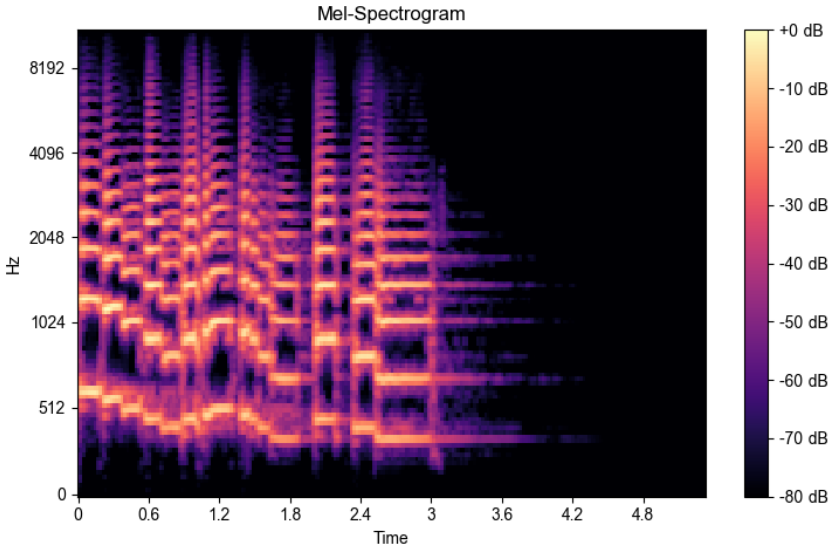
Timbre

- In the spectrogram, timbre shows fine structures of high frequencies that are not the main formants – such as rows of harmonics, fricatives, and breathing sounds.



Timbre

- Connected to Emotion:
  - The timbre of laughter and crying is distinctly different (even though the pitch levels are similar).
  - CNN models learn timbre patterns from Mel-Spectrograms to differentiate emotions, such as roughness/softness/ hoarseness.



Formants vs. Timbre

- In Voice AI (SER), both Formants and Timbre are sound properties reflected in the Spectrogram/Mel-Spectrogram, but they have different meanings and roles.

Feature	Formants	Timbre
Provide information about:	Vowel sounds, articulation, vocal cord muscle tension.	Individual characteristics, subtle emotions, vocal quality (harsh, high-pitched, hoarse).
Related to:	Tongue position, mouth shape, vowel sounds.	Harmonics, overtones, breath sounds, fricative sounds.
Visible of a spectrogram:	2-3 horizontal dark bands in the low-to-mid frequencies.	Detailed analysis across all frequencies, harmonic lines, and bandwidth.
Changes based on mood?:	Yes – Angry/Happy → higher formants, Sad → lower formants.	Yes – timbre changes (roughness/softness), fret noise increases/decreases.
Example:	The sound 'a' has a frequency range of F1~700Hz, F2~1200Hz; the sound 'ee' has a frequency range of F1~300Hz, F2~2300Hz.	John and Mary say "hello" using the same note – but the sound is different because of the timbre.

Why Mel-Spectrogram

- Dimensionality Reduction:
  A spectrogram of a 3-second audio file might be (513, 130), but a Mel-Spectrogram would only be (128, 130) → Faster computation, less memory usage, reduced chance of overfitting.
- Mimicking the human auditory system:

  Speech emotions (anger, sadness, joy) are perceived through the human ear.
  - Using a Mel-Spectrogram therefore provides features that allow CNNs to recognize emotional patterns better than using a linear spectrogram.

Why Mel-Spectrogram

- Emphasis on key elements (formants, pitch contour):

  Low frequencies on the Mel scale expand in detail, helping CNNs see the differences in vowel sounds and pitch, which are directly related to emotions.

Summary

- Spectrogram = An image showing frequency (Hz) versus time.
  - It shows high-frequency details well, suitable for engineering applications.
- Mel-Spectrogram = Adjusts the spectrogram to the scale that the human ear can actually hear → Provides better results in speech and emotion recognition applications with CNNs.

Summary

- Choose Mel-Spectrogram because:
  - It's smaller and more resource-efficient.
  - It emphasizes important areas (low-mid frequencies).
  - CNNs learn emotion patterns more easily.
  - To try it yourself, use the command librosa.example('libri1') and change n_mels to 64, 128, or 256, and see how the spectrogram changes.

Summary

- In Voice AI (SER), both Formants and Timbre are sound properties reflected in the Spectrogram/Mel-Spectrogram, but they have different meanings and roles.

Key	Description
Mel	Mel stands for Melody.
Mel scale	Mel scale is a frequency scale that mimics the human ear.
n_mels	n_mels is the number of rows (resolution) in the Mel-spectrogram.
Recommendation	Recommended values: 128 (default), 64 (for fast/soft tempo), 256 (high resolution).

Feature Extraction

MFCC Applications

- MFCC is one of the most popular feature extraction techniques for speech and emotion recognition because it mimics the workings of the human ear, which is more sensitive to frequencies in the Mel scale than to linear frequencies.

- MFCC stands for Mel-Frequency Cepstral Coefficients, a widely used feature extraction technique in audio and speech processing.
  - This technique transforms a raw audio signal into a series of numerical values (features) that record the key characteristics of the sound, particularly those related to human hearing.

MFCC Applications

- Mel-Frequency (Mel)
  - A frequency scale that mimics the perception of the human ear.
  - Humans distinguish low frequencies better than high frequencies.
- Cepstral
  - Derived from the word Spectral, it's a mathematical technique for separating the components of sound.
- Coefficients
  - Numerical values obtained from calculations.

MFCC Applications

- Feature extraction
  - The process of transforming raw data into a set of numerical values that a Machine Learning model can understand.
- Raw audio signal
  - Sound waves in their time form (waveform)
- Human hearing
  - The human ear's ability to perceive sound.

MFCC Applications

- Basic working principle:
  - Dividing the sound into smaller frames (e.g., 25 ms per frame).
  - Calculating a spectrogram for each frame.
  - Grouping frequencies according to the Mel Scale and calculating the energy within each group.
  - Using mathematical techniques (Discrete Cosine Transform – DCT) to extract coefficients that describe the overall spectrum.

- Applications in SER: MFCC values can indicate emotional characteristics of a sound; for example, sad sounds often have different energy frequencies than angry sounds.

MFCC Applications

- Example code for MFCC extraction using Librosa:

```
import librosa
import numpy as np

# File '1001_DFA_ANG_XX.wav' (angry). Specify the path to the data prepared in Colab.
file_path = 'path/to/your/Crema/1001_DFA_ANG_XX.wav'
y, sr = librosa.load(file_path, sr=16000) # Load the audio and set the sample rate.

# Calculate 13 MFCC values for each frame.
mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13, n_fft=512, hop_length=256)

print(f"Shape of MFCCs: {mfccs.shape}")
# Output: (13, frame), e.g., (13, 150) if the file is 2.4 seconds long

# We could use the average of each MFCC value throughout the file as a feature.
mfccs_mean = np.mean(mfccs, axis=1)
print(f"Mean MFCCs (13 features): {mfccs_mean}")
```

MFCC Applications

- Example code for MFCC extraction using Librosa: (teacher code)

```
import librosa
import numpy as np

# Let's say we have an audio file '1001_DFA_ANG_XX.wav' (angry sound).
file_path = '/mydrive/Crema/1/1001_DFA_ANG_XX.wav'
y, sr = librosa.load(file_path, sr=16000) # Load the audio and set the sample rate.

# Calculate 13 MFCC values for each frame.
mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13, n_fft=512, hop_length=256)

print(f"Shape of MFCCs: {mfccs.shape}")
# Output: (13, number of frames), e.g., (13, 150) if the file is 2.4 seconds long.

# We could use the average of each MFCC value throughout the file as a feature.
mfccs_mean = np.mean(mfccs, axis=1)
print(f"Mean MFCCs (13 features): {mfccs_mean}")
```

MFCC Applications

- The results of the MFCC extraction code using librosa: (teacher's code)

```
Shape of MFCCs: (13, 143)
Mean MFCCs (13 features): [-4.6292636e+02  9.4833954e+01  1.3507710e+01  2.4746634e+01
  8.9572840e+00 -5.5806642e+00 -1.1498092e+01 -7.6696334e+00
 -2.7332540e+00 -1.2431253e+01 -1.8618450e-01 -9.4540796e+00
 -5.5726290e+00]
```

MFCC Applications

- Shape: (13, 143)

```
Shape of MFCCs: (13, 143)
Mean MFCCs (13 features): [-4.6292636e+02  9.4833954e+01  1.3507710e+01  2.4746634e+01
  8.9572840e+00 -5.5806642e+00 -1.1498092e+01 -7.6696334e+00
 -2.7332540e+00 -1.2431253e+01 -1.8618450e-01 -9.4540796e+00
 -5.5726290e+00]
```

  - Dimension 1 (13) is the number of MFCC coefficients; the value obtained is 13.
    - This means we extract 13 coefficients per audio frame (as defined in n_mfcc=13).
    - Typically, MFCC 3 represents the total energy of the signal (sometimes called Log-Energy), and the 2nd-13th represent the characteristics of the spectrum.

MFCC Applications

- Shape: (13, 143)

```
Shape of MFCCs: (13, 143)
Mean MFCCs (13 features): [-4.6292636e+02  9.4833954e+01  1.3507710e+01  2.4746634e+01
  8.9572840e+00 -5.5806642e+00 -1.1498092e+01 -7.6696334e+00
 -2.7332540e+00 -1.2431253e+01 -1.8618450e-01 -9.4540796e+00
 -5.5726290e+00]
```

  - Dimension 2 (143) is the number of frames (time frames); the value is 143.
    - This means the audio file is divided into 143 sub-frames, calculated with hop_length=256 (moving in increments of 256 samples) at a sample rate of 16000 Hz.

MFCC Applications

- Shape: (13, 143)

```
Shape of MFCCs: (13, 143)
Mean MFCCs (13 features): [-4.6292636e+02  9.4833954e+01  1.3507710e+01  2.4746634e+01
  8.9572840e+00 -5.5806642e+00 -1.1498092e+01 -7.6696334e+00
 -2.7332540e+00 -1.2431253e+01 -1.8618450e-01 -9.4540796e+00
 -5.5726290e+00]
```

  - Dimension 2 (143) is the number of frames (time frames); the value is 143.
    - Each frame is 256/16000 = 0.016 seconds apart.
    - 143 frames is approximately 143 x 0.016 = 2.288 seconds in file length.

MFCC Applications

- Explanation of Mean MFCCs (13 values):
  - These values are the averages of each MFCC value throughout the file (averaged over 143 frames) to be used as a 13-value feature vector for this file.

Order	Scitific-value	Decimal	Meaning
MFCC 1	`-4.6292636e+02`	-462.93	A large negative logarithmic energy (Log Energy) indicates that the sound has relatively low energy (it might be normal speech, not very loud).
MFCC 2	`9.4833954e+01`	94.83	Overall is positive.
MFCC 3	`1.3507710e+01`	13.51	
MFCC 4	`2.4746634e+01`	24.75	
MFCC 5	`8.9572840e+00`	8.96	
MFCC 6	`-5.5806642e+00`	-5.58	Negative values start to show a difference.
MFCC 7	`-1.1498092e+01`	-11.50	
MFCC 8	`-7.6696334e+00`	-7.67	
MFCC 9	`-2.7332540e+00`	-2.73	
MFCC 10	`-1.2431253e+01`	-12.43	
MFCC 11	`-1.8618450e-01`	-0.19	Near the zero.
MFCC 12	`-9.4540796e+00`	-9.45	
MFCC 13	`-5.5726290e+00`	-5.57	

MFCC Applications

- In SER:
  - MFCC 1 indicates the loudness/energy of the sound.
    - It is the log energy of the sound signal, with the formula:
      - $C0 = \log(\Sigma x^2)$

Order	Scitific-value	Decimal	Meaning
MFCC 1	-4.6292636e+02	-462.93	A large negative logarithmic energy (Log Energy) indicates that the sound has relatively low energy (it might be normal speech, not very loud).
MFCC 2	9.4833954e+01	94.83	Overall is positive.
MFCC 3	1.3507710e+01	13.51	
MFCC 4	2.4746634e+01	24.75	
MFCC 5	8.9572840e+00	8.96	
MFCC 6	-5.5806642e+00	-5.58	Negative values start to show a difference.
MFCC 7	-1.1498092e+01	-11.50	
MFCC 8	-7.6696334e+00	-7.67	
MFCC 9	-2.7332540e+00	-2.73	
MFCC 10	-1.2431253e+01	-12.43	
MFCC 11	-1.8618450e-01	-0.19	Near the zero.
MFCC 12	-9.4540796e+00	-9.45	
MFCC 13	-5.5726290e+00	-5.57	

MFCC Applications

- In SER:
  - MFCC 1 indicates the loudness/energy of the sound.
    - Angry or happy sounds tend to have high energy (high MFCC 1 value).
    - Sad or neutral sounds have low energy (low MFCC 1 value).

Order	Scitific-value	Decimal	Meaning
MFCC 1	`-4.6292636e+02`	-462.93	A large negative logarithmic energy (Log Energy) indicates that the sound has relatively low energy (it might be normal speech, not very loud).
MFCC 2	`9.4833954e+01`	94.83	Overall is positive.
MFCC 3	`1.3507710e+01`	13.51	
MFCC 4	`2.4746634e+01`	24.75	
MFCC 5	`8.9572840e+00`	8.96	
MFCC 6	`-5.5806642e+00`	-5.58	Negative values start to show a difference.
MFCC 7	`-1.1498092e+01`	-11.50	
MFCC 8	`-7.6696334e+00`	-7.67	
MFCC 9	`-2.7332540e+00`	-2.73	
MFCC 10	`-1.2431253e+01`	-12.43	
MFCC 11	`-1.8618450e-01`	-0.19	Near the zero.
MFCC 12	`-9.4540796e+00`	-9.45	
MFCC 13	`-5.5726290e+00`	-5.57	

Each MFCC Member

- In SER:
  - The MFCC 1 value of −462.93 obtained is not typical, as Log Energy should normally be approximately:
    - Very loud sound: approximately 0 to +50 dB
    - Normal speech sound: approximately +20 to +40 dB
    - Low sound: approximately −10 to +10 dB

Order	Scitific-value	Decimal	Meaning
MFCC 1	−4.6292636e+02	-462.93	A large negative logarithmic energy (Log Energy) indicates that the sound has relatively low energy (it might be normal speech, not very loud).
MFCC 2	9.4833954e+01	94.83	Overall is positive.
MFCC 3	1.3507710e+01	13.51	
MFCC 4	2.4746634e+01	24.75	
MFCC 5	8.9572840e+00	8.96	
MFCC 6	−5.5806642e+00	-5.58	Negative values start to show a difference.
MFCC 7	−1.1498092e+01	-11.50	
MFCC 8	−7.6696334e+00	-7.67	
MFCC 9	−2.7332540e+00	-2.73	
MFCC 10	−1.2431253e+01	-12.43	
MFCC 11	−1.8618450e−01	-0.19	Near the zero.
MFCC 12	−9.4540796e+00	-9.45	
MFCC 13	−5.5726290e+00	-5.57	

Preliminary Analysis

- From the MFCC values obtained:
  - MFCC 1 is very low (-462.93), indicating that this file may contain normal or low-pitched speech (not screaming or a very loud sound).
  - The value obtained is in a different decibel format (possibly dBFS – Decibels relative to Full Scale).
  - Or it may be a value that has been normalized.

Order	Scitific-value	Decimal	Meaning
MFCC 1	-4.6292636e+02	-462.93	A large negative logarithmic energy (Log Energy) indicates that the sound has relatively low energy (it might be normal speech, not very loud).
MFCC 2	9.4833954e+01	94.83	Overall is positive.
MFCC 3	1.3507710e+01	13.51	
MFCC 4	2.4746634e+01	24.75	
MFCC 5	8.9572840e+00	8.96	
MFCC 6	-5.5806642e+00	-5.58	Negative values start to show a difference.
MFCC 7	-1.1498092e+01	-11.50	
MFCC 8	-7.6696334e+00	-7.67	
MFCC 9	-2.7332540e+00	-2.73	
MFCC 10	-1.2431253e+01	-12.43	
MFCC 11	-1.8618450e-01	-0.19	Near the zero.
MFCC 12	-9.4540796e+00	-9.45	
MFCC 13	-5.5726290e+00	-5.57	

Each MFCC Member

- In SER:
  - MFCC 2–4: Indicate the spectral slope, which shows how quickly energy decreases as the frequency increases.
  - For example:
    - Male–female voice (pitch)
    - Bass–high pitch (pitch)

Order	Scitific-value	Decimal	Meaning
MFCC 1	`-4.6292636e+02`	-462.93	A large negative logarithmic energy (Log Energy) indicates that the sound has relatively low energy (it might be normal speech, not very loud).
MFCC 2	`9.4833954e+01`	94.83	Overall is positive.
MFCC 3	`1.3507710e+01`	13.51	
MFCC 4	`2.4746634e+01`	24.75	
MFCC 5	`8.9572840e+00`	8.96	
MFCC 6	`-5.5806642e+00`	-5.58	Negative values start to show a difference.
MFCC 7	`-1.1498092e+01`	-11.50	
MFCC 8	`-7.6696334e+00`	-7.67	
MFCC 9	`-2.7332540e+00`	-2.73	
MFCC 10	`-1.2431253e+01`	-12.43	
MFCC 11	`-1.8618450e-01`	-0.19	Near the zero.
MFCC 12	`-9.4540796e+00`	-9.45	
MFCC 13	`-5.5726290e+00`	-5.57	

Each MFCC Member

- In SER:
  - MFCC 5–13: Describe the fine structure of the spectrum, such as the position of formants, which is very important for speech.

Order	Scitific-value	Decimal	Meaning
MFCC 1	`-4.6292636e+02`	-462.93	A large negative logarithmic energy (Log Energy) indicates that the sound has relatively low energy (it might be normal speech, not very loud).
MFCC 2	`9.4833954e+01`	94.83	Overall is positive.
MFCC 3	`1.3507710e+01`	13.51	
MFCC 4	`2.4746634e+01`	24.75	
MFCC 5	`8.9572840e+00`	8.96	
MFCC 6	`-5.5806642e+00`	-5.58	Negative values start to show a difference.
MFCC 7	`-1.1498092e+01`	-11.50	
MFCC 8	`-7.6696334e+00`	-7.67	
MFCC 9	`-2.7332540e+00`	-2.73	
MFCC 10	`-1.2431253e+01`	-12.43	
MFCC 11	`-1.8618450e-01`	-0.19	Near the zero.
MFCC 12	`-9.4540796e+00`	-9.45	
MFCC 13	`-5.5726290e+00`	-5.57	

Each MFCC Member

- In SER:
  - MFCC 5–13: Describe the specific details of voice that relate to various emotions, such as:
    - Vibration,
    - Hoarseness,
    - Complexity.

Order	Scitific-value	Decimal	Meaning
MFCC 1	`-4.6292636e+02`	-462.93	A large negative logarithmic energy (Log Energy) indicates that the sound has relatively low energy (it might be normal speech, not very loud).
MFCC 2	`9.4833954e+01`	94.83	Overall is positive.
MFCC 3	`1.3507710e+01`	13.51	
MFCC 4	`2.4746634e+01`	24.75	
MFCC 5	`8.9572840e+00`	8.96	
MFCC 6	`-5.5806642e+00`	-5.58	Negative values start to show a difference.
MFCC 7	`-1.1498092e+01`	-11.50	
MFCC 8	`-7.6696334e+00`	-7.67	
MFCC 9	`-2.7332540e+00`	-2.73	
MFCC 10	`-1.2431253e+01`	-12.43	
MFCC 11	`-1.8618450e-01`	-0.19	Near the zero.
MFCC 12	`-9.4540796e+00`	-9.45	
MFCC 13	`-5.5726290e+00`	-5.57	

Parameter: n_mfcc

n_mfcc	General Use	Reasoning
13	Standard value (default)	Most frequently used in speech recognition (SR) because the first 13 values cover most of the important information.
20	General audio analysis	Add more detail to the model.
26	Music Information Retrieval task	Music has a more complex range of frequencies than words.
40	Deep Learning task	Feed the data into a CNN/LSTM model for self-learning.
12-15	SER (Speech Emotion Recognition) task	13 is the optimal value.
8-10	High-speed work	Reducing the dimensionality of data speeds up training, but may reduce quality.
50-60	Specialized research	Requires very high detail.

ค่าของ n_mfcc

n_mfcc	Features	Precision	Speed	Task
8	8	Coarse	High-Speed	Real-time system, embedded devices
13	13	Moderate	Good	Speech recognition, SER
20	20	Fine	Moderate	Music analysis, Speaker ID
26	26	Refined	Low	Music genre classification
40	40	Granular	Very Low	Deep learning, research

References

- ศิริพงษ์ กลิ่นขจร, Accessing on April 29, 2026, https://nerdoptimize.com/ai/what-is-deep-learning/#generative-adversarial-networks-gans
- Put Dejudom, Accessing on April 29, 2026, https://www.linkedin.com/pulse/เบองตนของ-convolutional-neural-networkscnn-put-dejudom/

References

Reference Materials

- MediaPipe: https://mediapipe.readthedocs.io/en/latest/solutions/hands.html, accessed August 18, 2025
- MediaPipe Solutions Guide: https://ai.google.dev/edge/mediapipe/solutions/guide, accessed August 25, 2025
- Video Technology: https://www.coastalwiki.org/wiki/Video_technology, accessed August 25, 2024
- Digital Video: https://en.wikipedia.org/wiki/Digital_video, accessed August 25, 2024
- H.264 vs. MJPEG in Embedded Vision: All You Need to Know, https://www.technexion.com/resources/h-264-vs-mjpeg-in-embedded-vision-all-you-need-to-know/, Accessed on Aug 31, 2025

Mahidol University